AGENTIC AI FOR DISASTER RESPONSE

DisasterOps

Intelligent Shelter Coordination and Emergency Logistics System

Logical, Mathematical & Technical System Specification

A human-controlled Agentic AI framework for real-time shelter monitoring, resource allocation, logistics planning, and dynamic disaster response.

PROJECT

VJAI Hackathon 2026

THEME

Agentic AI for Sustainable Goals

DOCUMENT STATUS

Prototype Technical Specification

VERSION

v1.0

TEAM

Special Needs Dev Team

DisasterOps: Decision Logic and Framework - Main PDF

Contents

1 DisasterOps Decision Logic 1
1.1 Purpose . . . 1
2 Main AI Decision Loop 1
3 Step 1: Information Intake 2
4 Step 2: Validation 3
5 Step 3: Duplicate Detection 4
6 Step 4: Data Completeness 5
7 Urgency Calculation Method 5
7.1 Overview . . . 5
8 Factor 1: Time Pressure 6
8.1 Calculating Time To Failure . . . 7
8.2 Clock and State Updates . . . 8
9 Factor 2: Resource Severity 8
10 Factor 3: Vulnerability 9
11 Factor 4: Population Affected 10
12 Factor 5: Lack of Local Coping Alternatives 10
12.1 More Detailed Local Coping Model . . . 11
13 Complete Urgency Equation 12
14 Urgency Categories 12
15 Worked Example 13
16 Worked Example: Separation From Candidate ETA 14
17 Data Confidence 14
17.1 Data Completeness . . . 15
17.2 Data Freshness . . . 15
17.3 Source Reliability . . . 15
17.4 Low-Confidence Behavior . . . 16
18 Hard Safety Overrides 16
19 Comparing Active Needs 17
20 Generating Feasible Options 17
21 Resource Constraint 18
22 Route Feasibility 18
23 Re-planning Logic 19

24 Example of Alternative Reallocation 20
25 Human Approval 21
26 Monitoring 21
27 Re-planning Trigger 22
28 Division of Responsibilities Between AI and Deterministic Algorithms 22
28.1 LLM / Agent . . . 22
28.2 Deterministic Code . . . 23
28.3 RAG . . . 23
28.4 Human Coordinator . . . 23
29 Prototype Assumptions 24
30 User Input and Shelter Status Reporting 24
30.1 Purpose . . . 24
31 Input Categories 25
32 1. Report Metadata 25
33 2. Shelter Population 25
34 3. Vulnerable Population 26
35 4. Resource Inventory 27
35.1 Consumption Rate . . . 27
35.2 Time To Failure . . . 28
36 5. Infrastructure Status 28
37 6. Transportation and Shelter Accessibility 29
38 7. Active Resource Requests 30
39 8. Incoming Deliveries 30
40 9. Surplus Resources 31
41 10. Operational Problems 31
42 11. Changes Since Previous Report 32
43 12. Data Confidence Input 32
44 13. Evidence Attachments 32
45 14. Emergency Update 33
46 15. Hourly Form Example 33
47 16. Mapping User Input to DisasterOps Variables 35
48 17. Automatic Inputs 35
49 18. Input Design Principle 36
50 Feasibility Scoring Framework 36
50.1 Purpose . . . 36
50.2 Feasibility Score Definition . . . 37
51 Hard Feasibility Constraints 38
52 Feasibility Factors 39
52.1 Factor Summary . . . 39
53 Factor 1: Route Feasibility 39
54 Factor 2: ETA Margin 40
55 Factor 3: Vehicle Feasibility 42
56 Factor 4: Resource Availability 43

57 Factor 5: Road Data Reliability and Freshness 44
58 Factor 6: Mission Complexity 45
59 Factor 7: Approval Readiness 46
60 Factor 8: Weather and Obstruction Safety 46
61 Factor 9: Alternative Source Quality 47
62 Factor 10: Data Confidence 48
63 Complete Feasibility Equation 49
64 Feasibility Status and Categories 49
65 Worked Example: Direct Delivery 50
66 Worked Example: Road Closure 52
67 Worked Example: Cross-Shelter Transfer 53
68 Pre-Authorized Contingency Plans 54
69 Missing Data Behaviour 54
70 Plan Feasibility Algorithm 55
71 Relationship Between Urgency and Feasibility 56
72 Important Limitation 57
73 Next Required Module 58
74 Candidate Plan Generation Framework 59
74.1 Purpose . . . 59
75 Candidate Generator Inputs 60
75.1 Target Need . . . 60
75.2 Shared Operational State . . . 61
76 Candidate Plan Object 61
77 Service Level Generation 62
77.1 Full Fulfillment Quantity . . . 63
77.2 Target Inventory . . . 63
77.3 Minimum Stabilization Quantity . . . 63
77.4 Quantity Candidate Set . . . 64
78 Stage 1: Source Discovery 64
78.1 Transferable Supply . . . 64
78.2 Cross-Shelter Transfer . . . 65
79 Stage 2: Vehicle Discovery 65
79.1 Basic Vehicle Filter . . . 66
79.2 Vehicle Capacity . . . 66
80 Stage 3: Personnel Availability 66
81 Stage 4: Preliminary Geographic Screening 67
81.1 Stage 4A: Route Matrix Screening . . . 67
81.2 Vehicle-to-Source Travel . . . 68
82 Total Candidate ETA 68
83 Stage 5: Preliminary Candidate Pruning 69
83.1 Cheap Pre-Filters . . . 69
83.2 Top-K Source Screening . . . 70
83.3 Top-K Vehicle Screening . . . 70
84 Stage 6: Detailed Google Maps Route Generation 70
84.1 Traffic-Aware Routing . . . 71

85 DisasterOps Road State Layer **71**
85.1 Route Overlay Check . . . 72
85.2 Why a Separate Road Layer Is Required . . . 72
86 Custom Road Closure Representation **72**
87 Stage 7: Weather Integration **73**
87.1 Weather Inputs . . . 73
87.2 Weather Should Be Checked Along the Route . . . 74
87.3 Time-Dependent Weather . . . 74
87.4 Weather Risk Conversion . . . 74
87.5 Weather Alerts . . . 75
88 Stage 8: Candidate Construction Patterns **75**
88.1 Pattern A: Direct Warehouse Delivery . . . 75
88.2 Pattern B: Alternative Warehouse Delivery . . . 76
88.3 Pattern C: Cross-Shelter Transfer . . . 76
88.4 Pattern D: Delayed Vehicle Candidate . . . 76
88.5 Pattern E: Multi-Trip Candidate . . . 77
88.6 Multi-Trip Inventory Simulation . . . 77
88.7 Pattern F: Split Delivery Candidate . . . 78
88.8 Split Delivery Limitation . . . 78
88.9 Pattern G: Partial Stabilization Candidate . . . 78
88.10 Pattern H: Mission Reassignment Candidate . . . 79
89 Candidate Generation and Active Missions **79**
90 Stage 9: SOP and RAG Checks **80**
91 Stage 10: Preliminary Candidate Disposition **81**
92 Stage 11: Candidate Deduplication **81**
93 Stage 12: Candidate Diversity Preservation **82**
94 Maximum Candidate Count **83**
95 Candidate Pruning Heuristic **83**
96 Route Candidate Generation Strategy **83**
97 Why Two Routing Stages Are Used **84**
98 Weather API Efficiency Strategy **85**
99 Candidate Generation Algorithm **85**
100 Candidate Generation Output Example **90**
101 Candidate Generation During Replanning **93**
102 Partial Replanning **94**
103 Google Maps Integration Contract **95**
104 Weather Integration Contract **96**
105 Candidate Generation Tool Contract **96**
106 Candidate Generation Safety Principles **98**
106.1 Principle 1: Never Invent Resources . . . 98
106.2 Principle 2: Never Invent Road Safety . . . 98
106.3 Principle 3: Never Invent Missing Quantity . . . 98
106.4 Principle 4: Never Hide Partial Fulfillment . . . 98
106.5 Principle 5: Never Create Hidden Downstream Risk . . . 99
106.6 Principle 6: Candidate Generation Must Remain Explainable . . . 99

107 Candidate Generation MVP Scope **99**
107.1 Not Required for Initial MVP . . . 100
108 Candidate Generation Example After Road Closure **100**
109 Relationship to Feasibility and Utility **101**
109.1 Candidate Generation . . . 101
109.2 Feasibility . . . 101
109.3 Utility Ranking . . . 101
110 Complete Planning Pipeline **102**
111 Next Required Module **103**
112 Plan Utility and Ranking Framework **104**
112.1 Purpose . . . 104
113 Important Design Principle **105**
114 Feasibility Gate Before Utility **105**
115 Utility Score Definition **106**
116 Prototype Utility Formula **106**
116.1 Factor Weights . . . 107
117 Factor 1: System Impact **107**
117.1 Normalized Urgency . . . 107
117.2 Normalized Unmet Need . . . 108
117.3 Need Burden . . . 108
117.4 Total System Burden . . . 108
117.5 Before and After Simulation . . . 109
117.6 Why System Impact Is Important . . . 110
118 Evaluation Horizon **110**
119 Factor 2: Effective Need Coverage **111**
119.1 Multi-Delivery Candidate . . . 111
119.2 Late Delivery Discount . . . 111
119.3 Interpretation . . . 112
119.4 Effective Delivered Quantity . . . 112
119.5 Example . . . 113
120 Factor 3: Downstream Safety **113**
120.1 Remaining Buffer . . . 113
120.2 Plan Downstream Safety . . . 114
120.3 Hard Safety Rule . . . 114
121 Factor 4: Existing Mission Continuity **114**
121.1 Mission Delay . . . 114
121.2 Mission Slack . . . 115
121.3 Disruption Penalty . . . 115
121.4 Critical Mission Override . . . 115
122 Factor 5: Operational Efficiency **116**
122.1 Vehicle-Time Cost . . . 116
122.2 Distance Cost . . . 116
122.3 Trip Cost . . . 116
122.4 Load Utilization . . . 117
122.5 Efficiency Score . . . 117
123 Factor 6: Future Operational Optionality **117**

123.1 Vehicle Reserve . . . 117
123.2 Resource Reserve . . . 118
123.3 Optionality Score . . . 118
124 Complete Utility Equation 118
125 Utility Categories 118
126 Worked Example 119
126.1 Candidate P1: Central Warehouse . . . 119
126.2 Candidate P2: Nearby Shelter Transfer . . . 120
126.3 Candidate P3: Vehicle Reassignment . . . 121
127 Final Ranking Example 122
128 Pareto Dominance Filter 123
129 Utility Ranking Algorithm 123
130 Tie-Breaking Rules 127
131 No Feasible High-Utility Candidate 127
132 Road Closure Re-Ranking 128
133 Google Maps and Weather in Utility Ranking 129
134 Avoiding Double Counting 129
135 Human-Readable Recommendation Output 130
136 Utility Explanation Object 131
137 Ranking Output 132
138 Human Approval 132
139 Simultaneous Critical Needs 133
140 Prototype Simplification 134
141 Complete DisasterOps Decision Brain 135
142 Core Mathematical Architecture Summary 138
143 Prototype Validation Requirement 139
144 Normative Implementation Contracts 140
144.1 Purpose and Requirement Language . . . 140
144.2 Decision Context . . . 140
144.3 Scalar, Unit, and Nullability Conventions . . . 141
144.4 Core Enumerations . . . 141
144.5 Urgency Result Contract . . . 142
144.6 Candidate Plan Contract . . . 142
144.7 Feasibility Result Contract . . . 143
144.8 Utility and Recommendation Contract . . . 143
144.9 Configuration Source of Truth . . . 144
144.10 Event Envelope, Idempotency, and Ordering . . . 145
144.11 Atomic Reservation and Approval Commit . . . 145
144.12 Normative Invariants . . . 146
144.13 Minimum Acceptance Tests . . . 147
144.14 Implementation Milestones and Completion Gates . . . 149
145 Implementation Map and Web Application Architecture 149
145.1 Purpose . . . 149
146 Why a Web Application 150
147 User Roles 151

147.1 Role 1: Shelter Volunteer / Shelter Manager 151
147.2 Role 2: Municipal Coordinator 151
147.3 Optional Future Role: Driver / Logistics Staff 152
148 Authentication 152
148.1 User Object 153
149 Role-Based Access Control 154
150 Volunteer User Interface 154
150.1 Volunteer Home Screen 155
151 Volunteer Hourly Status Form 156
151.1 Volunteer Critical Update 157
152 Volunteer Supply Request Screen 158
153 Volunteer Delivery Screen 159
154 Coordinator User Interface 160
154.1 Coordinator Dashboard 160
155 Coordinator Map 162
155.1 Map Interaction 162
156 Coordinator AI Recommendation Screen 163
157 Coordinator Approval Workflow 165
158 Coordinator Modify Plan 165
159 Coordinator Shelter Detail Page 166
160 Frontend Route Structure 166
161 Frontend Component Map 167
162 Backend Module Architecture 168
163 Core Backend Function Map 169
164 API Endpoint Map 169
165 Example Shelter Report JSON 170
166 Example Need Object 171
167 Example Candidate Plan Object 172
168 Example Recommendation Object 173
169 Database Tables 173
170 Core Database Relationships 174
171 Shared State Service 175
172 Decision Engine API 176
173 Agent Orchestration Layer 178
173.1 Example Agent Workflow 179
174 External Service Integration Map 181
175 Event-Driven Replanning Architecture 181
176 Notification System 182
177 Low Connectivity and Offline Behaviour 183
178 Data Conflict After Offline Synchronization 183
179 Audit Logging 184
180 System Status Flow 185
181 Suggested Prototype Technology Stack 187
182 Minimum Hackathon Implementation 188

183 Recommended Demo User Flow 189
184 Final Implementation Principle 191

1 DisasterOps Decision Logic

1.1 Purpose

DisasterOps is an Agentic AI system designed to assist human emergency coordinators in managing multiple evacuation shelters during a disaster.

The system does not replace human decision-makers. Its purpose is to continuously:

1. collect and normalize information from shelters,
2. detect missing, duplicated, outdated, or conflicting information,
3. maintain a shared operational state,
4. calculate the urgency of active needs,
5. compare requests across multiple shelters,
6. generate feasible resource-allocation options,
7. check logistics constraints such as roads, vehicles, supply availability, and SOP requirements,
8. recommend a plan,
9. request human approval when required,
10. monitor execution,
11. and automatically re-plan when conditions change.

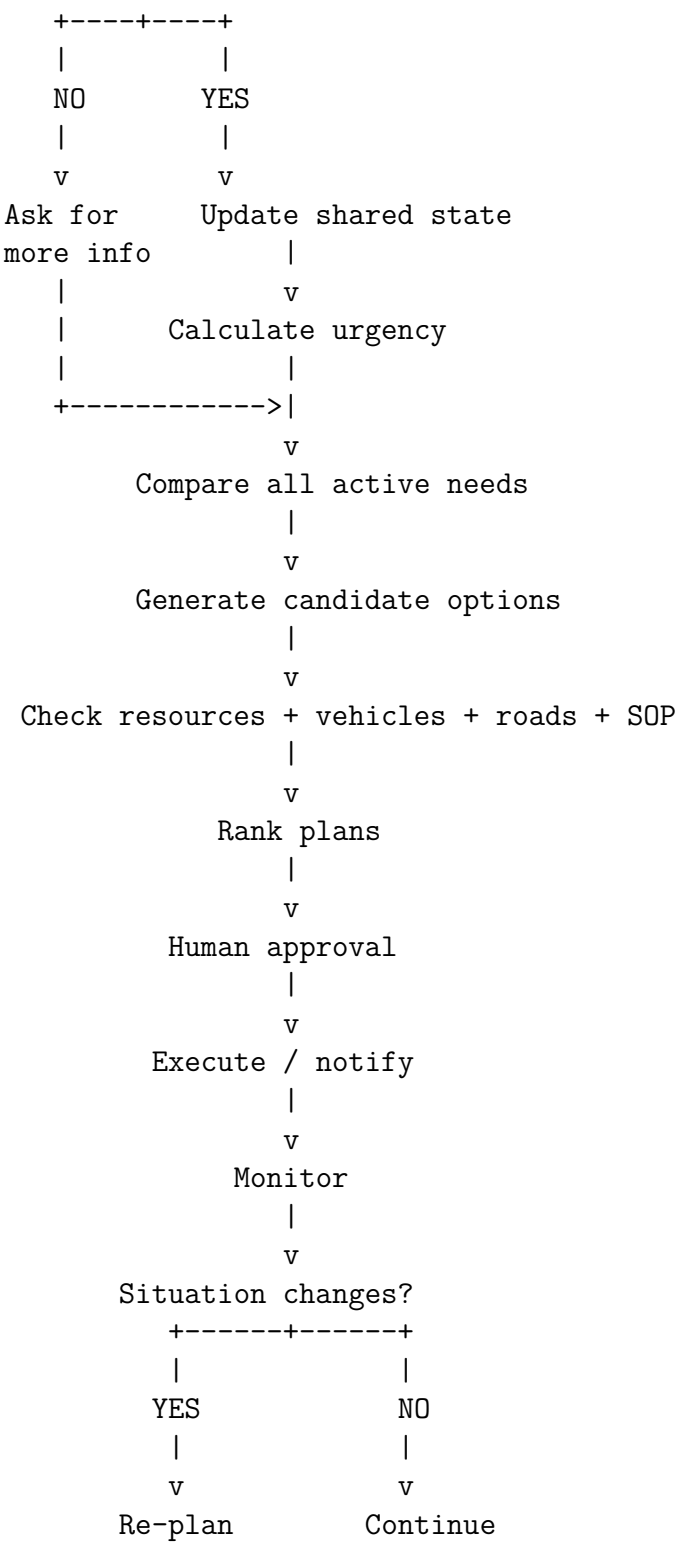
The fundamental principle is:

> The AI should not simply answer questions. It should maintain an understanding of the current situation, recognize when the situation changes, gather additional information when necessary, and continuously update its proposed plan.

2 Main AI Decision Loop

The main operational loop of DisasterOps is:

```
New information arrives
        |
        v
Understand + validate
        |
        v
Duplicate?
        |
        v
Information complete?
```



3 Step 1: Information Intake

Whenever new information enters the system, DisasterOps first converts it into a structured format.

Possible information sources include:

- shelter staff reports,

- municipality staff,
- inventory databases,
- vehicle status,
- road closure reports,
- weather information,
- emergency-management databases,
- voice messages,
- text messages,
- forms,
- sensor feeds,
- official emergency announcements.

Each shelter should have a continuously updated state.
For shelter i, define its state as:

$$X_i = \{N_i, N_{v,i}, Q_{i,r}, C_{i,r}, t_i, L_i, R_i, E_i, \ldots\}$$

where:

- N_i = total number of evacuees,
- $N_{v,i}$ = number of vulnerable evacuees,
- $Q_{i,r}$ = remaining quantity of resource r,
- $C_{i,r}$ = estimated consumption rate of resource r,
- t_i = timestamp of the most recent update,
- L_i = shelter location,
- R_i = active requests,
- E_i = additional emergency conditions.

4 Step 2: Validation

Before information is used for decision-making, DisasterOps validates it.
The system should check:

1. Is the report associated with a known shelter?
2. Does the report contain a valid timestamp?
3. Is the data recent enough?

4. Does the request duplicate another active request?
5. Does the new information contradict existing information?
6. Are essential fields missing?

For example:

Shelter A reports: “Water is running low.”

This statement alone is not sufficient.
The system may require:

- current amount of water,
- shelter population,
- estimated consumption rate,
- time of last inventory measurement.

If these values are unavailable, DisasterOps should request additional information before making a high-confidence decision.

5 Step 3: Duplicate Detection

Multiple staff members may report the same problem.
Example:

Report 1: “Shelter A needs drinking water urgently.”
Report 2: “We are almost out of water at Shelter A.”

The system should determine whether these represent the same underlying request.
Possible factors include:

- same shelter,
- same resource category,
- similar timestamps,
- semantic similarity of text,
- similar requested quantity.

If two reports are considered duplicates, they should be merged into a single active need rather than generating two separate deliveries.

6 Step 4: Data Completeness

Before urgency is calculated, the system checks whether sufficient information exists.

For a resource request, the minimum required information may include:

- resource type,
- current quantity,
- population affected,
- estimated consumption rate,
- timestamp,
- shelter location.

Additional information may include:

- vulnerable population,
- alternative supply sources,
- current road access,
- vehicle availability.

If critical information is missing, DisasterOps should explicitly recognize:

"I do not currently have enough information to calculate urgency reliably."

The agent then requests the missing information.

7 Urgency Calculation Method

7.1 Overview

Each active resource need receives an urgency score:

$$U \in [1, 10]$$

where:

- $U = 1$ represents very low urgency,
- $U = 10$ represents extreme urgency.

The preliminary DisasterOps urgency model is:

$$U = 1 + 9\,(0.40T + 0.20S + 0.15V + 0.15P + 0.10A)$$

where all individual components are normalized to the interval:

$$[0, 1].$$

Therefore:

$$1 \leq U \leq 10.$$

The five factors are:

$$T = \text{Time Pressure}$$

$$S = \text{Resource Severity}$$

$$V = \text{Vulnerability}$$

$$P = \text{Population Affected}$$

$$A = \text{Lack of Alternatives}$$

8 Factor 1: Time Pressure

Time pressure receives the largest weight because the most important need-level question is:

How soon will this need reach its critical threshold if no intervention occurs?

Time pressure must be intrinsic to the need. It must not depend on a route, source, vehicle, or candidate-plan ETA because those objects do not exist until after candidate generation. Candidate-specific arrival timing is evaluated later by the Feasibility Engine as the ETA Margin.

Define:

$$TTF = \text{Time To Failure}$$

and let:

$$\tau_r = \text{configured critical-time scale for resource } r.$$

The prototype time-pressure function is:

$$T = \begin{cases} 1, & TTF \leq 0, \\ \dfrac{1}{1 + \left(\frac{TTF}{\tau_r}\right)^{\kappa_r}}, & TTF > 0, \end{cases}$$

where $\kappa_r > 0$ controls how sharply urgency rises as failure approaches. Both τ_r and κ_r are versioned configuration values, not values invented by the LLM.

This function has the required behavior:

$$TTF \to 0^+ \Rightarrow T \to 1,$$

$$TTF = \tau_r \Rightarrow T = 0.5,$$

$$TTF \gg \tau_r \Rightarrow T \to 0.$$

The exact shape must be calibrated against SOPs and historical scenarios. The architecture requirement is the separation of intrinsic time pressure from candidate-specific arrival timing.

8.1 Calculating Time To Failure

For a consumable resource:

$$TTF = \frac{Q - Q_{\text{reserve}}}{C}$$

where:

- Q = current resource quantity,
- Q_{reserve} = minimum safety reserve,
- C = estimated consumption rate per hour.

The deterministic implementation MUST handle edge cases explicitly:

$$TTF = \begin{cases} \dfrac{Q - Q_{\text{reserve}}}{C}, & C > 0, \\ +\infty, & C = 0 \text{ and } Q > Q_{\text{reserve}}, \\ 0, & C = 0 \text{ and } Q \leq Q_{\text{reserve}}. \end{cases}$$

A negative consumption rate is `INVALID_INPUT`. If Q, Q_{reserve}, or C is unknown, TTF is null and Urgency returns `NEEDS_DATA` unless an independent hard safety override applies.

If no safety reserve is used in the prototype:

$$TTF = \frac{Q}{C}.$$

For example, suppose:

$$Q = 240 \text{ litres}$$

and

$$C = 80 \text{ litres/hour}.$$

Therefore:

$$TTF = \frac{240}{80} = 3 \text{ hours}.$$

For the prototype example, use:

$$\tau_{water} = 3 \text{ hours}$$

and:

$$\kappa_{water} = 2.$$

Then:

$$T = \frac{1}{1 + (3/3)^2} = 0.5.$$

This result depends only on the shelter need and the configured resource policy. A nearby warehouse and a distant warehouse therefore produce the same need-level urgency; their different ETAs are evaluated on their respective candidate plans.

8.2 Clock and State Updates

Urgency is dynamic because TTF changes as resources are consumed and because new shelter reports may change inventory, consumption, population, severity, vulnerability, or local coping capacity.

If one hour passes with no delivery and consumption remains constant, the example becomes:

$$TTF = 2 \text{ hours}$$

Then:

$$T = \frac{1}{1 + (2/3)^2} \approx 0.692.$$

A road closure by itself does not change T. It invalidates or degrades candidate plans, changes their ETAs, and may trigger replanning. Urgency is recalculated only when need state or evaluation time changes.

9 Factor 2: Resource Severity

Different shortages have different consequences.

Define:

$$S \in [0, 1].$$

A prototype severity table could be:

Resource / Condition	Severity S
Non-essential comfort item	0.10
Hygiene supplies	0.20
Blankets in normal temperature	0.30
General batteries / charging	0.50
Food	0.60
Drinking water	0.80
Heating during dangerous cold	0.90
Critical essential power	1.00

These numbers are prototype values.

In a real deployment, severity values must be determined using:

- official disaster-management SOPs,

- government guidelines,
- municipality policy,
- historical disaster data,
- emergency-management experts.

The LLM should not arbitrarily invent the severity value.
The value should come from an explicit policy or rule table.

10 Factor 3: Vulnerability

The system should account for shelters containing vulnerable populations.
Define:

$$q_v = \frac{N_v}{N}$$

where:

- N_v = number of vulnerable evacuees,
- N = total shelter population.

A preliminary normalized vulnerability score is:

$$V = \min\left(1, \frac{q_v}{q_{\text{ref}}}\right)$$

where:

$$q_{\text{ref}} = 0.30$$

is a prototype reference value.
Therefore:

$$V = \min\left(1, \frac{N_v}{0.30N}\right).$$

This equation applies only when $N > 0$. If $N = 0$ and $N_v = 0$, define $V = 0$ because no population is currently affected. If $N_v > N$, either count is negative, or $N = 0$ while $N_v > 0$, return `INVALID_INPUT`. Unknown population values remain null.
Example:

$$N = 80$$

and

$$N_v = 12.$$

Then:

$$V = \frac{12}{0.30(80)} = \frac{12}{24} = 0.50.$$

This means vulnerability contributes half of its maximum possible component.
The exact definition of "vulnerable" must eventually follow official emergency-management policy rather than being invented by the AI.

11 Factor 4: Population Affected

The urgency score should consider how many people are affected.

However, population should not scale linearly.

Otherwise a very large shelter with a relatively minor problem could always outrank a small shelter experiencing an immediate critical shortage.

Therefore DisasterOps uses a logarithmic population function.

Define:

$$P = \min\left(1, \frac{\ln(1+N)}{\ln(1+N_{\text{ref}})}\right).$$

For the prototype:

$$N_{\text{ref}} = 500.$$

Therefore:

$$P = \min\left(1, \frac{\ln(1+N)}{\ln(501)}\right).$$

For example, if:

$$N = 80,$$

then:

$$P = \frac{\ln(81)}{\ln(501)} \approx 0.71.$$

The logarithmic function gives meaningful importance to population while preventing extremely large shelters from dominating the urgency calculation purely because of size.

12 Factor 5: Lack of Local Coping Alternatives

A request becomes more urgent when the shelter has little ability to cope locally while waiting for external assistance. This factor is limited to alternatives already present at or controlled by the affected shelter, such as a verified backup tank, generator, on-site reserve, or immediately usable local substitute.

It must not depend on candidate warehouses, vehicles, external routes, or delivery ETAs. Those alternatives are discovered only after urgency calculation and are evaluated by Candidate Generation, Feasibility, and Utility.

Define:

$$A \in [0, 1]$$

where:

- $A = 0$ means strong verified local coping capacity exists,
- $A = 1$ means no verified local coping capacity exists.

A simple prototype scale may be:

Local Coping Situation	A
Several verified on-site substitutes or reserves	0.10
One strong verified local fallback	0.30
Partial local fallback with limited duration	0.60
Weak or unverified local fallback	0.80
No verified local fallback	1.00

12.1 More Detailed Local Coping Model

For a more quantitative version, each local fallback j can receive a coping-quality score:

$$B_j = K_j R_j e^{-\Delta t_j / \tau_{local}}$$

where:

- K_j = fraction of required resource that local fallback j can provide,
- R_j = reliability of the local fallback,
- Δt_j = time until the local fallback becomes usable,
- τ_{local} = configured local-delay sensitivity parameter.

The best verified local fallback is:

$$B = \max_j (B_j).$$

Then:

$$A = 1 - B.$$

If no verified local fallback exists:

$$B = 0$$

and therefore:

$$A = 1.$$

If a highly reliable, high-capacity, immediately usable local fallback exists:

$$B \approx 1$$

and:

$$A \approx 0.$$

13 Complete Urgency Equation

The complete prototype model is therefore:

$$\boxed{U = 1 + 9\left[0.40T + 0.20S + 0.15V + 0.15P + 0.10A\right]}$$

with:

$$T = \begin{cases} 1, & TTF \leq 0, \\ \dfrac{1}{1 + \left(\frac{TTF}{\tau_r}\right)^{\kappa_r}}, & TTF > 0 \end{cases}$$

$$TTF = \frac{Q - Q_{\text{reserve}}}{C}$$

$$V = \min\left(1, \frac{N_v}{0.30N}\right)$$

$$P = \min\left(1, \frac{\ln(1+N)}{\ln(501)}\right)$$

and:

$$A = 1 - B$$

where B represents the quality of the best verified local coping alternative. Candidate-plan ETA is deliberately absent from this equation.

14 Urgency Categories

For user-interface purposes, the continuous score may be converted into categories:

Urgency Score	Category
1.0–2.9	Low
3.0–4.9	Moderate
5.0–6.9	High
7.0–8.9	Very High
9.0–10.0	Critical

Internally, DisasterOps should retain the full deterministic value for stable comparisons. The user interface should show the category, at most one decimal place, the data-confidence level, and the prototype-model label.

Example:

$$U_{internal} = 8.4327$$

rather than simply storing:

$$\text{Very High (8.4/10)}, \quad \text{Confidence: 71\%}.$$

This avoids false precision in the interface while preserving deterministic ordering internally.

15 Worked Example

Consider Shelter A.

- Population: $N = 80$
- Vulnerable population: $N_v = 12$
- Resource: drinking water
- Time to failure: $TTF = 3$ hours
- Configured $\tau_{water} = 3$ hours
- Configured $\kappa_{water} = 2$
- Strong verified local coping capacity exists

Then:

$$T = \frac{1}{1 + (3/3)^2} = 0.50$$

For drinking water:

$$S = 0.80.$$

For vulnerability:

$$V = \frac{12}{0.30(80)} = 0.50.$$

Population:

$$P = \frac{\ln(81)}{\ln(501)} \approx 0.71.$$

Suppose:

$$A = 0.30.$$

Then:

$$U = 1 + 9[0.40(0.50) + 0.20(0.80) + 0.15(0.50) + 0.15(0.71) + 0.10(0.30)].$$

Therefore:

$$U \approx 1 + 9(0.5715)$$

$$U \approx 6.14.$$

Thus:

$$\boxed{U \approx 6.1/10}.$$

The request has high intrinsic urgency and should enter candidate generation.

16 Worked Example: Separation From Candidate ETA

Suppose the state above produces two candidate plans:

- Plan P1: nearby warehouse, $ETA = 0.5$ hours,
- Plan P2: distant warehouse, $ETA = 2.5$ hours.

Both plans target the same need. Therefore both reference the same need-level result:

$$U \approx 6.1/10.$$

Their timing differs in the Feasibility Engine:

$$E_{P1} = 1$$

and:

$$E_{P2} = \text{clamp}\left(\frac{3 - 2.5}{0.5(3)}, 0, 1\right) \approx 0.333.$$

If a road closure invalidates P1, its status becomes `HARD_INFEASIBLE`. The closure does not instantly rewrite the need's intrinsic urgency. The system removes P1, re-evaluates P2 and any new candidates, and continues to update TTF as time passes.

One hour later, if no aid has arrived and all other need variables are unchanged:

$$TTF = 2 \text{ hours}$$

and:

$$T = \frac{1}{1 + (2/3)^2} \approx 0.692.$$

The intrinsic urgency now increases because the resource runway decreased:

$$\boxed{U \approx 6.8/10}.$$

This separation prevents route difficulty from being counted once in urgency and again in feasibility.

17 Data Confidence

Urgency and data confidence must be treated separately.

A shelter with outdated information should not automatically receive a low urgency score.

Define:

$$C_D \in [0, 1]$$

as the Data Confidence score.

A prototype confidence model is:

$$C_D = 0.50M + 0.30F + 0.20R$$

where:

$$M = \text{data completeness}$$

$$F = \text{data freshness}$$

$$R = \text{source reliability.}$$

17.1 Data Completeness

If k required variables exist out of K required variables:

$$M = \frac{k}{K}.$$

17.2 Data Freshness

Data freshness may decay exponentially:

$$F = e^{-t/\tau_f}$$

where:

- t = age of the data,
- τ_f = configurable freshness constant.

For rapidly changing information such as road status or shelter inventory, τ_f should be relatively small.

17.3 Source Reliability

Example prototype values could be:

Source	Reliability R
Official municipality system	1.00
Verified shelter manager	0.90
Emergency responder	0.90
Registered volunteer	0.70
Unverified public report	0.40
Unknown source	0.20

These are prototype values only.

17.4 Low-Confidence Behavior

If:

$$C_D < 0.50,$$

the urgency score should be displayed as provisional.
For example:

Urgency: 7.4/10 – Provisional

Data Confidence: 42%

Reason: current shelter inventory has not been confirmed recently.

Action: request immediate status update.

If essential information is missing, the system may display:

Urgency: Unknown

Action: additional information required.

This is safer than interpreting missing information as evidence that no problem exists.

18 Hard Safety Overrides

Certain situations should not rely entirely on the weighted urgency equation.
Hard rules should exist.
For example:

$$TTF \leq 0$$

indicates that the critical threshold has already been reached.
The system may therefore enforce:

$$U \geq 9.$$

Candidate-specific timing must not override the need-level urgency equation. After candidate generation, if every candidate satisfies:

$$ETA_p \geq TTF$$

for a life-safety resource with:

$$S \geq 0.8,$$

the planner must emit a separate operational escalation flag:

`NO_TIMELY_INTERVENTION`

This flag triggers immediate human escalation and external-resource search. It does not retroactively inject ETA into U.

Critical life-safety situations should be escalated immediately to a human emergency coordinator rather than handled only through an automated numerical score.

The AI must never allow a weighted average to override an explicit emergency-management rule.

19 Comparing Active Needs

After calculating urgency, DisasterOps compares all currently active requests.

Requests are first sorted by:

$$U$$

from highest to lowest.

Example:

Shelter	Need	Urgency
Shelter C	Critical power	9.2
Shelter A	Drinking water	8.4
Shelter D	Food	7.1
Shelter B	Blankets	4.3

If two requests have similar urgency scores, tie-breakers may be applied in the following order:

1. lower Time To Failure,
2. higher resource severity,
3. higher vulnerable population,
4. larger population affected,
5. older unresolved request.

20 Generating Feasible Options

High urgency does not automatically mean a plan is feasible.

For each request, DisasterOps searches for possible actions.

Potential options include:

- send supplies from the central warehouse,
- transfer surplus resources from another shelter,
- use a different vehicle,
- use an alternative road,
- combine multiple deliveries,
- delay a lower-priority delivery,
- escalate to the human coordinator.

For each option j, the system must verify:

- sufficient resource quantity,

- vehicle capacity,
- vehicle availability,
- road feasibility,
- ETA,
- consequences for the source shelter,
- compliance with SOPs,
- whether human approval is mandatory.

21 Resource Constraint

For a delivery of resource quantity q:

$$q \leq Q_{\text{available}}.$$

If a vehicle has capacity:

$$K_v,$$

then:

$$q \leq K_v.$$

If resources are transferred from Shelter B to Shelter A, DisasterOps must ensure that Shelter B remains above its own safety reserve:

$$Q_B - q \geq Q_{B,\text{reserve}}.$$

The system should never solve one shortage by creating another critical shortage.

22 Route Feasibility

A route is feasible only if all required road segments are currently considered usable.

If route R contains segments:

$$R = \{e_1, e_2, \ldots, e_n\},$$

then:

$$\text{Feasible}(R) = 1$$

only if:

$$\text{Open}(e_i) = 1$$

for every required segment.

If any segment becomes unavailable:

$$\text{Open}(e_k) = 0,$$

the current route becomes invalid.
This triggers the re-planning loop.

23 Re-planning Logic

Suppose DisasterOps creates:

Van 3: Shelter B → Shelter A

Deliver: 100 L water

ETA: 15 minutes

A new event arrives:

Road B–A is blocked.

The system performs:

1. identify missions affected by the road closure,
2. invalidate the old route,
3. calculate alternative routes,
4. update ETA,
5. recalculate candidate ETA margin and feasibility,
6. check alternative supply sources,
7. compare revised options,
8. generate a new plan,
9. recalculate intrinsic urgency only if need state or evaluation time changed,
10. request approval if required.

Suppose:

$$ETA_{\text{old}} = 15 \text{ min}$$

and:

$$ETA_{\text{new}} = 55 \text{ min}.$$

DisasterOps then compares:

$$ETA_{\text{new}}$$

against:

$$TTF.$$

If:

$$ETA_{\text{new}} < TTF,$$

rerouting may remain acceptable.
If:

$$ETA_{\text{new}} \geq TTF,$$

the system must search for another solution.

24 Example of Alternative Reallocation

Suppose Shelter A has:

$$TTF = 45 \text{ min.}$$

The rerouted delivery would take:

$$55 \text{ min.}$$

Therefore:

$$ETA > TTF.$$

The current plan is unacceptable.
DisasterOps then evaluates:

- another vehicle,
- another warehouse,
- another shelter with surplus water,
- reprioritization of another active mission.

For example:

Option 1: Continue current route.

ETA = 55 min.

Result: predicted late arrival.

Reject.

Option 2: Transfer 70 L from Shelter C using Van 1.

ETA = 21 min.

Shelter C remains above reserve.

Feasible.

Therefore DisasterOps recommends Option 2.

25 Human Approval

DisasterOps is a decision-support system.

High-impact operational decisions remain under human control.

The system should provide:

- recommended action,
- urgency score,
- reasoning,
- resource calculations,
- route information,
- alternative plans,
- relevant SOP evidence,
- uncertainty,
- expected consequences.

The coordinator may then:

- approve,
- modify,
- reject,
- request more information.

26 Monitoring

After a plan is approved, DisasterOps continues monitoring the mission.

Possible state changes include:

- road closure,
- vehicle breakdown,
- delayed delivery,
- change in shelter population,
- new resource request,
- updated inventory,
- new vulnerable evacuees,
- supply source becoming unavailable,
- mission completion.

Any relevant change causes the system to check whether the existing plan is still valid.

27 Re-planning Trigger

Define the current plan as:

$$\Pi_t.$$

When a new event E_{t+1} changes the operational state:

$$X_t \to X_{t+1},$$

the system checks:

$$\text{Valid}(\Pi_t | X_{t+1}).$$

If:

$$\text{Valid}(\Pi_t | X_{t+1}) = 0,$$

the system initiates re-planning:

$$\Pi_t \to \Pi_{t+1}.$$

This continuous observe–evaluate–plan–monitor–replan loop is one of the main characteristics that makes DisasterOps an Agentic AI system rather than a conventional chatbot.

28 Division of Responsibilities Between AI and Deterministic Algorithms

The LLM should not calculate everything.

DisasterOps should use different tools for different tasks.

28.1 LLM / Agent

The AI agent may:

- understand natural-language reports,
- recognize missing information,
- identify potentially duplicated reports,
- determine which tool should be called,
- retrieve relevant SOPs,
- interpret changing situations,
- generate possible actions,
- explain recommendations.

28.2 Deterministic Code

Ordinary algorithms should calculate:

- Time To Failure,
- consumption rates,
- urgency score,
- remaining inventory,
- vehicle capacity,
- route feasibility,
- distance,
- ETA,
- alternative-resource availability.

28.3 RAG

Retrieval-Augmented Generation should provide:

- municipality disaster SOPs,
- shelter procedures,
- resource-priority rules,
- vulnerable-person policies,
- emergency-management guidelines.

28.4 Human Coordinator

Humans remain responsible for:

- approving high-impact resource movements,
- overriding incorrect recommendations,
- handling exceptional situations,
- approving escalation,
- making decisions beyond the system's authorized scope.

29 Prototype Assumptions

The equations and weights defined in this document are preliminary engineering assumptions for the DisasterOps prototype.

The model:

$$0.40T + 0.20S + 0.15V + 0.15P + 0.10A$$

has not yet been clinically, operationally, or governmentally validated.

Before real-world deployment, the following must be performed:

1. validation against official emergency-management SOPs,
2. consultation with disaster-response experts,
3. calibration using historical disaster cases,
4. sensitivity analysis of weights,
5. testing against simulated emergency scenarios,
6. bias and failure-mode analysis,
7. validation of escalation thresholds.

The prototype equation is therefore intended as an explainable starting model rather than a final emergency-management standard.

30 User Input and Shelter Status Reporting

30.1 Purpose

DisasterOps requires a continuously updated operational state for every active shelter.

The primary human input is an **Hourly Shelter Status Report**, completed by an authorized shelter volunteer or shelter manager.

The form should not require the user to re-enter all information every hour.

Instead:

- previous values are automatically pre-filled,
- the user confirms whether each value is unchanged,
- only changed information must be updated,
- timestamps are recorded automatically,
- critical changes may be reported immediately without waiting for the next hourly report.

The hourly report serves three purposes:

1. maintain an accurate shared operational state,
2. provide the variables required for urgency calculations,
3. detect changes that may require re-planning.

31 Input Categories

Each hourly report consists of the following major categories:

1. Report Metadata
2. Shelter Population
3. Vulnerable Population
4. Resource Inventory
5. Infrastructure Status
6. Transportation and Accessibility
7. Active Requests
8. Incoming and Outgoing Deliveries
9. Operational Problems
10. Data Confidence
11. Additional Notes and Evidence

32 1. Report Metadata

The following information identifies the report.

Field	**Description**
Shelter ID	Unique identifier assigned to the shelter
Shelter Name	Official or temporary shelter name
Shelter Location	GPS coordinates or registered address
Reporter ID	ID of authorized volunteer or shelter manager
Reporter Role	Volunteer, shelter manager, municipality staff, etc.
Report Timestamp	Automatically generated by the system
Last Successful Report	Timestamp of previous confirmed report
Report Type	Hourly / Emergency Update / Correction

Shelter ID, location, and registered shelter information should normally be stored in the system and not manually re-entered every hour.

33 2. Shelter Population

The system must know how many people currently depend on the shelter.

Required fields:

Field	Variable	Required?
Current total evacuees	N	Yes
New arrivals since last report	N_{in}	Yes
Departures since last report	N_{out}	Yes
Expected incoming evacuees	N_{expected}	Optional
Shelter maximum capacity	$N_{\max}$	Pre-registered

The system verifies consistency using:

$$N_t \approx N_{t-1} + N_{\text{in}} - N_{\text{out}}.$$

Large inconsistencies should trigger a confirmation request.

34 3. Vulnerable Population

The system should use aggregate counts rather than unnecessary personal identifying information.

Possible categories include:

- elderly evacuees,
- people with mobility limitations,
- people requiring accessibility assistance,
- pregnant evacuees,
- infants or very young children,
- people dependent on electrically powered essential equipment,
- other municipality-defined priority groups.

Define:

$$N_v = \sum_{k=1}^{m} N_{v,k}.$$

The form may contain:

Category	Count
Elderly evacuees	Input
Mobility assistance required	Input
Pregnant evacuees	Input
Infants / very young children	Input
Essential electrically dependent needs	Input
Other priority group defined by SOP	Input

These categories must eventually follow local emergency-management policy. DisasterOps should not independently define medical vulnerability.

35 4. Resource Inventory

For every important resource r, the shelter reports the current available quantity:

$$Q_r.$$

Possible resource categories include:

- drinking water,
- food,
- blankets,
- hygiene supplies,
- batteries,
- generator fuel,
- emergency lighting,
- heating resources,
- cooling resources,
- sanitation supplies,
- other municipality-defined essentials.

For every tracked resource, the form should contain:

Field	**Description**
Current quantity	Amount physically available now
Unit	litres, meals, units, kg, etc.
Estimated hourly consumption	If known
Minimum safety reserve	Defined by SOP or system policy
Last physically verified	Timestamp
Quantity confidence	Measured / Estimated / Unknown
Expected delivery	Quantity currently on the way
Expected delivery ETA	Current ETA

35.1 Consumption Rate

For resource r:

$$C_r = \frac{\Delta Q_r}{\Delta t}.$$

If sufficient historical observations exist, DisasterOps may estimate consumption automatically.

For example:

$$C_{\text{water}} = \frac{Q_{\text{water},t-1} - Q_{\text{water},t}}{\Delta t}.$$

If no reliable historical estimate exists, the system may use a predefined planning assumption from the relevant SOP.

The source of the assumption must be recorded.

35.2 Time To Failure

For resource r:

$$TTF_r = \frac{Q_r - Q_{\text{reserve},r}}{C_r}.$$

This represents the estimated amount of time before the resource falls below its safety reserve.

If:

$$Q_r \leq Q_{\text{reserve},r},$$

then:

$$TTF_r \leq 0.$$

This condition requires immediate escalation.

36 5. Infrastructure Status

The shelter must report whether essential infrastructure is functioning.

Fields include:

Infrastructure	**Possible Status**
Electricity	Normal / Limited / Failed
Backup generator	Available / Running / Failed / None
Generator fuel	Quantity or estimated runtime
Drinking-water access	Available / Limited / Failed
Sanitation / toilets	Normal / Limited / Failed
Heating	Normal / Limited / Failed
Cooling / ventilation	Normal / Limited / Failed
Lighting	Normal / Limited / Failed
Mobile network	Normal / Weak / Unavailable
Internet connection	Normal / Weak / Unavailable
Building access	Normal / Restricted / Unsafe

A critical infrastructure change is treated as a new event and may trigger immediate re-planning.

37 6. Transportation and Shelter Accessibility

DisasterOps must understand whether resources can physically reach the shelter.

The shelter report should include:

- primary access road status,
- secondary access road status if known,
- vehicle access possible,
- pedestrian-only access,
- debris or obstruction,
- flooding,
- bridge damage,
- loading/unloading accessibility,
- approximate obstruction location,
- confidence in the road report.

Road-status values may include:

$$\{\text{Open}, \text{Restricted}, \text{Blocked}, \text{Unknown}\}.$$

If the volunteer reports:

"Road to Shelter A is blocked by debris."

DisasterOps must:

1. identify the relevant road segment,
2. update the shared transport state,
3. identify missions using that road,
4. invalidate affected routes,
5. request route recalculation,
6. recalculate affected ETAs,
7. recalculate intrinsic urgency only when need state or evaluation time changed.

38 7. Active Resource Requests

A shelter may submit an explicit resource request.

Each request should contain:

Field	Description
Requested resource	Water, food, blankets, etc.
Requested quantity	Desired quantity
Current available quantity	Current inventory
Reason for request	Short explanation
Requested by	Reporter / shelter manager
Request timestamp	Automatically generated
Human perceived urgency	Optional Low–Critical selection
Additional notes	Free text

The human-reported urgency does not directly become the DisasterOps urgency score. Instead it acts as additional evidence.

The system independently calculates:

$$U.$$

A large difference between human-reported urgency and calculated urgency should be flagged for review.

39 8. Incoming Deliveries

For every expected delivery:

- source location,
- resource type,
- quantity,
- assigned vehicle,
- departure time,
- current ETA,
- current mission status,
- route status.

Possible mission states include:

$$\{\text{Planned}, \text{Approved}, \text{Dispatched}, \text{In Transit}, \text{Delayed}, \text{Arrived}, \text{Completed}, \text{Cancelled}\}.$$

When a delivery arrives, the shelter confirms:

- actual quantity received,

- arrival time,
- discrepancies,
- damaged or unusable supplies.

The shelter inventory is then updated automatically.

40 9. Surplus Resources

Shelters should report resources that exceed their required safety reserve.

Define available transferable surplus:

$$Q_{\text{surplus},r} = \max\left(0, Q_r - Q_{\text{reserve},r}\right).$$

However, DisasterOps should not automatically transfer the entire surplus.

A further operational safety margin may be required:

$$Q_{\text{transferable},r} = \max\left(0, Q_r - Q_{\text{reserve},r} - Q_{\text{buffer},r}\right).$$

This allows the system to identify shelters capable of supporting another shelter without creating a new shortage.

41 10. Operational Problems

The hourly form should include a section asking whether any new operational problems have occurred.

Examples:

- overcrowding,
- shortage of volunteers,
- loss of power,
- water interruption,
- sanitation failure,
- heating/cooling failure,
- blocked access road,
- damaged shelter area,
- communication outage,
- transport unavailable,
- unexpected increase in evacuees,
- incoming supply delay,
- other.

Each reported problem becomes an event that may update the shared state.

42 11. Changes Since Previous Report

To minimize user burden, volunteers should be presented with the previous report.

The form asks:

"Has anything significantly changed since the previous report?"

Possible answer:

$$\{\text{No Change}, \text{Minor Change}, \text{Major Change}\}.$$

If "No Change" is selected, the volunteer confirms the pre-filled values.

If "Major Change" is selected, DisasterOps may immediately trigger a full state review.

43 12. Data Confidence Input

For important quantitative values, the volunteer should indicate how the information was obtained.

Possible confidence types:

- physically counted,
- digitally measured,
- estimated by shelter staff,
- reported by another person,
- unknown.

Example:

Water remaining: 200 L

Measurement method: Estimated

Last checked: 14:20

This information contributes to the Data Confidence score C_D.

44 13. Evidence Attachments

Where useful, volunteers may attach:

- photographs,
- short videos,
- voice recordings,
- documents,
- map pins.

Examples include:

- photograph of blocked road,
- photograph of damaged shelter entrance,
- photograph of remaining supply stock,
- voice report when typing is impractical.

Attachments should supplement, not replace, structured fields.

45 14. Emergency Update

The hourly form must not be the only communication mechanism.
A separate:

CRITICAL UPDATE

function should allow shelter staff to immediately report major changes.
Examples:

- critical resource exhausted,
- road suddenly blocked,
- shelter loses electricity,
- shelter becomes inaccessible,
- evacuation required,
- major unexpected population increase,
- essential infrastructure failure.

Critical updates bypass the normal hourly reporting schedule and immediately enter the DisasterOps decision loop.

46 15. Hourly Form Example

A simplified hourly report displayed to a volunteer may appear as follows:

```
SHELTER A - 14:00 STATUS UPDATE

1. Population
Current evacuees: [ 82 ]
New arrivals last hour: [ 4 ]
Departures last hour: [ 2 ]

2. Vulnerable Groups
```

Elderly: [12]
Mobility assistance: [3]
Pregnant evacuees: [1]
Infants / young children: [5]
Essential electrically dependent needs: [1]

3. Water
Current amount: [210] litres
Measured / Estimated: [Estimated]
Last physically checked: [13:45]

4. Food
Current meals available: [390]
Last checked: [13:50]

5. Other Supplies
Blankets: [95]
Batteries: [32]
Hygiene kits: [40]

6. Infrastructure
Electricity: [Normal]
Generator: [Available]
Water supply: [Limited]
Sanitation: [Normal]
Heating: [Normal]
Mobile network: [Weak]

7. Access
Primary road: [Open]
Secondary road: [Unknown]
Vehicle access: [Yes]

8. Active Needs
[ ] No new request
[X] New request

Resource: [Drinking Water]
Requested quantity: [300 L]
Reason:
[Current stock is decreasing faster than expected.]

9. Deliveries
Expected delivery: [200 L Water]
Vehicle: [Van 3]
ETA: [38 min]

10. Changes

```
Any major change since previous report?
[ No ]

11. Notes
[ _____________________________ ]

CONFIRM AND SUBMIT
```

47 16. Mapping User Input to DisasterOps Variables

The following table defines which form fields directly feed the decision model.

Input	**Variable**	**Use**
Total evacuees	N	Population factor
Vulnerable evacuees	N_v	Vulnerability factor
Current resource quantity	Q_r	Resource state
Consumption rate	C_r	Time To Failure
Safety reserve	$Q_{\text{reserve},r}$	Time To Failure
Resource runway Delivery ETA	TTF ETA_p	Intrinsic Time Pressure Candidate ETA Margin / Feasibility
Resource type	S	Severity
Alternative sources	A	Alternative availability
Road status	—	Routing and ETA
Vehicle status	—	Feasibility
Timestamp	—	Data freshness
Measurement method	—	Data confidence

48 17. Automatic Inputs

Not all DisasterOps data should come from volunteers.

Where possible, the system should automatically retrieve:

- current time,
- shelter location,
- weather conditions,
- road network,
- known road closures,
- vehicle GPS positions,
- calculated route ETA,
- inventory records,
- previous shelter reports,

- municipality SOPs,
- existing active missions.

Manual reporting should focus on information that cannot reliably be obtained automatically.

49 18. Input Design Principle

The system should minimize the cognitive burden placed on shelter volunteers during a disaster.

Therefore the design principle is:

> Collect the minimum amount of human input required to maintain a sufficiently accurate operational state.

The app should prefer:

- pre-filled fields,
- one-tap confirmation,
- automatic timestamps,
- dropdown selections,
- structured resource units,
- voice-to-structured-input conversion,
- automatic calculations.

Free-text fields should primarily be used for exceptional circumstances that cannot be represented by structured fields.

50 Feasibility Scoring Framework

50.1 Purpose

The Urgency Score and Feasibility Score represent two fundamentally different questions.

- **Urgency Score:** How urgently does this shelter need require intervention?
- **Feasibility Score:** How realistically can this specific candidate plan be executed?

For example:

```
Shelter A Urgency = 9.5 / 10

Plan A Feasibility = 3.2 / 10
Plan B Feasibility = 8.8 / 10
Plan C Feasibility = HARD INFEASIBLE
```

A highly urgent situation must not cause an impossible logistical plan to receive a high feasibility score.

The system therefore follows the sequence:

```
Candidate Plan
      |
      v
Hard Constraint Check
      |
      +---- constraint violated ----> HARD INFEASIBLE
      |
      v
Calculate normalized feasibility factors
      |
      v
Calculate weighted Feasibility Score
      |
      v
Return:
- Feasibility Score
- Hard constraints
- Weak factors
- Explanation
```

50.2 Feasibility Score Definition

For every candidate plan p that is valid, sufficiently specified, and not blocked by a hard constraint, DisasterOps assigns:

$$F_p \in [1, 10]$$

where:

- $F = 1$ represents very low scored feasibility,
- $F = 10$ represents extremely feasible.

Feasibility status and Feasibility Score are separate fields. Before any weighted score is calculated, the candidate must first pass structural checks, required-data checks, and all **hard constraints**.

Therefore:

$$\text{If any hard constraint fails:} \qquad F = \varnothing$$

and the plan is marked:

$$\boxed{\text{HARD INFEASIBLE}}$$

If essential data are missing:

$$F = \varnothing, \qquad \text{status} = \texttt{NEEDS_DATA}.$$

The value $\varnothing$ must be serialized as JSON `null`, never as zero or 1.0. This prevents a blocked plan from being confused with a valid but weak plan.

A weighted average must never compensate for a violated physical or safety constraint.

51 Hard Feasibility Constraints

A candidate plan becomes hard infeasible if any of the following conditions occur.

1. No valid route exists.
2. The selected route contains a road segment confirmed as blocked.
3. A required road segment does not exist in the current system state.
4. The assigned vehicle does not exist.
5. The assigned vehicle is not operational.
6. The assigned vehicle cannot carry the planned load:

   $$K_v < L_p$$

   where:

   - K_v = vehicle capacity,
   - L_p = planned trip load.

7. The selected supply source does not contain sufficient transferable resources.
8. A resource transfer would reduce the source below its required safety reserve.
9. An official travel prohibition applies.
10. Required human approval has explicitly been denied.
11. ETA exceeds an explicit operational hard deadline defined by an SOP or coordinator.
12. Input data are structurally invalid, for example:

    - negative inventory,
    - negative ETA,
    - negative vehicle capacity,
    - zero or negative planned delivery quantity,
    - impossible confidence values,
    - invalid road status.

Missing essential data are not a physical constraint violation. They produce `NEEDS_DATA`; structurally invalid input produces `INVALID_INPUT`; a confirmed physical, policy, or authorization violation produces `HARD_INFEASIBLE`.

52 Feasibility Factors

Plans that pass all hard constraints receive a weighted feasibility score.

The preliminary DisasterOps model uses:

$$F_{\text{raw}} = 0.16R + 0.17E + 0.11V + 0.13S + 0.09D + 0.07C + 0.05A + 0.06W + 0.04B + 0.12Q$$

where:

$$0 \leq R, E, V, S, D, C, A, W, B, Q \leq 1$$

and:

$$F = 1 + 9F_{\text{raw}}$$

Therefore:

$$1 \leq F \leq 10.$$

52.1 Factor Summary

Factor	**Symbol**	**Weight**
Route Feasibility	R	0.16
ETA / Time Margin	E	0.17
Vehicle Readiness / Capacity Margin	V	0.11
Transferable Resource Headroom	S	0.13
Road Data Reliability	D	0.09
Mission Complexity	C	0.07
Approval Readiness	A	0.05
Weather / Obstruction Safety	W	0.06
Alternative Source Quality	B	0.04
Data Confidence	Q	0.12
Total		**1.00**

All coefficients are **prototype engineering assumptions**. They must later be validated or calibrated using official SOPs, historical disaster cases, simulations, and emergency-management experts.

The factors are not statistically independent. In particular, route passability, road-data reliability, weather risk, ETA margin, and overall data confidence may be correlated. Therefore F is a deterministic ranking heuristic, not a probability of mission success. The system must never label $F = 8.8$ as "88% likely to succeed." Sensitivity tests should vary correlated inputs together.

53 Factor 1: Route Feasibility

Each road segment is assigned a passability value.

Prototype values:

Road Status	Passability Value
Open	1.00
Restricted	0.65
Unknown	0.35
Blocked	0.00

For a route consisting of segments:

$$r_1, r_2, \ldots, r_n$$

the route feasibility factor is:

$$R = \min_i \left(\text{Passability}(r_i)\right)$$

The minimum is used because the route is limited by its weakest required segment.
Example:

$$1.00,\ 1.00,\ 0.65$$

gives:

$$R = 0.65.$$

If any required road segment is:

$$\text{BLOCKED}$$

then the candidate plan is immediately:

$$\boxed{\text{HARD INFEASIBLE}}.$$

An unknown road is not automatically considered blocked.
Instead:

$$R = 0.35$$

and confidence in the plan is also reduced.

54 Factor 2: ETA Margin

The ETA factor measures whether the proposed intervention is expected to arrive with sufficient time remaining before failure.

Define:

$$ETA = \text{Estimated Time of Arrival}$$

and:

$$TTF = \text{Time To Failure}.$$

The prototype ETA factor is:

$$E = \text{clamp}\left(\frac{TTF - ETA}{0.5TTF}, 0, 1\right)$$

where:

$$\text{clamp}(x, 0, 1) = \max\left(0, \min(1, x)\right).$$

This formula applies only when $TTF > 0$. If $TTF \leq 0$, define $E = 0$ and emit the applicable threshold-breached escalation. If $ETA < 0$, return `INVALID_INPUT`.

Interpretation:

$$ETA \leq 0.5TTF \Rightarrow E = 1$$

$$ETA = 0.75TTF \Rightarrow E = 0.5$$

$$ETA \geq TTF \Rightarrow E = 0.$$

Example:

$$TTF = 120 \text{ min}$$

$$ETA = 40 \text{ min}$$

then:

$$E = \text{clamp}\left(\frac{120 - 40}{60}, 0, 1\right)$$

$$E = 1.$$

Another plan:

$$ETA = 90 \text{ min}$$

then:

$$E = \frac{120 - 90}{60} = 0.5.$$

If:

$$ETA = 130 \text{ min}$$

then:

$$E = 0.$$

If TTF is unknown, the prototype should not assume either good or bad timing. When timing is essential under the resource policy, return `NEEDS_DATA`. Otherwise a temporary neutral value may be used:

$$E = 0.5$$

while the Data Confidence factor is reduced and an explicit `TTF_UNKNOWN` uncertainty code is stored.

If an explicit hard deadline exists and:

$$ETA > Deadline$$

then the plan becomes:

$$\boxed{\text{HARD INFEASIBLE}}.$$

55 Factor 3: Vehicle Feasibility

Vehicle feasibility consists of:

1. vehicle operational status,
2. vehicle availability,
3. vehicle capacity.

Define:

$$t_v = \text{minutes until vehicle becomes available.}$$

Vehicle availability is:

$$V_{\text{availability}} = e^{-t_v/\tau_v}$$

where the prototype uses:

$$\tau_v = 30 \text{ min.}$$

Examples:

$$t_v = 0 \Rightarrow V_{\text{availability}} = 1.00$$

$$t_v = 15 \Rightarrow V_{\text{availability}} \approx 0.607$$

$$t_v = 30 \Rightarrow V_{\text{availability}} \approx 0.368.$$

Capacity is first enforced as a hard constraint. For plans that pass, capacity margin measures robustness to loading error:

$$V_{\text{margin}} = \text{clamp}\left(\frac{K_v - L_p}{h_v L_p}, 0, 1\right)$$

where:

- K_v = vehicle capacity,
- L_p = planned trip load,
- h_v = configured capacity-headroom reference, prototype $h_v = 0.25$.

The vehicle factor is:

$$V = V_{\text{availability}} V_{\text{margin}}.$$

A plan that exactly fills the reported vehicle capacity still passes the hard constraint but receives low margin because small measurement or packing errors could make execution fail. This factor must not be used to reject a plan unless a separate policy threshold requires headroom.

However, if:

$$K_v < L_p$$

the candidate itself is invalid.

It should not receive a low score.

Instead:

$$\boxed{\text{HARD INFEASIBLE}}.$$

If multiple trips are possible, the plan generator must generate a separate candidate representing those multiple trips.

56 Factor 4: Resource Availability

For a source s, define:

$$Q_{\text{onhand}} = \text{current inventory}$$

$$Q_{\text{reserve}} = \text{required safety reserve}$$

$$Q_{\text{committed}} = \text{inventory already assigned to other missions.}$$

Transferable inventory is:

$$Q_{\text{transferable}} = \max\left(0, Q_{\text{onhand}} - Q_{\text{reserve}} - Q_{\text{committed}}\right).$$

If the plan requests:

$$Q_{\text{plan}},$$

the hard resource constraint is:

$$Q_{\text{transferable}} \geq Q_{\text{plan}}.$$

If it passes, the scored factor measures transferable headroom:

$$S = \text{clamp}\left(\frac{Q_{\text{transferable}} - Q_{\text{plan}}}{h_s Q_{\text{plan}}}, 0, 1\right),$$

where h_s is a configured headroom reference, prototype $h_s = 0.50$. If:

$$Q_{\text{transferable}} < Q_{\text{plan}},$$

the candidate plan is:

$$\boxed{\text{HARD INFEASIBLE}}.$$

The system must always enforce:

$$Q_{\text{source after transfer}} \geq Q_{\text{reserve}}.$$

This prevents DisasterOps from solving one shortage by creating another shortage at the donor location.

57 Factor 5: Road Data Reliability and Freshness

Route feasibility and route information reliability are different concepts.

Route feasibility asks:

> According to current data, can the vehicle travel through this road?

Road reliability asks:

> How much should DisasterOps trust that road information?

For road segment i:

$$F_i = e^{-a_i/\tau_r}$$

where:

- a_i = age of road information in minutes,
- τ_r = freshness decay constant.

Prototype:

$$\tau_r = 60 \text{ min}.$$

If source reliability is:

$$R_i^{source},$$

then road-information quality is:

$$q_i = R_i^{source} F_i.$$

The total route data-quality factor is:

$$D = 0.70 \min_i(q_i) + 0.30 \frac{\sum_i t_i q_i}{\sum_i t_i}$$

where t_i is the expected travel time on road segment i.

This gives strong importance to the least reliable segment while still considering the overall route.

58 Factor 6: Mission Complexity

More operational steps create more potential failure points.

Define:

$$N_{\text{extra stops}} = \max(0, N_{\text{stops}} - 1).$$

Then:

$$C = e^{-0.25N_{\text{extra stops}} - 0.45N_{\text{transfers}}}.$$

Examples:

Direct delivery:

$$N_{\text{stops}} = 1$$

$$N_{\text{transfers}} = 0$$

therefore:

$$C = 1.$$

For:

$$N_{\text{stops}} = 2$$

and:

$$N_{\text{transfers}} = 0,$$

then:

$$C = e^{-0.25} \approx 0.779.$$

For:

$$N_{\text{stops}} = 3$$

and:

$$N_{\text{transfers}} = 1,$$

then:

$$C = e^{-0.95} \approx 0.387.$$

This does not mean complex missions are forbidden.

It means that, all else equal, a simpler mission is more operationally reliable.

59 Factor 7: Approval Readiness

Approval readiness is defined as:

$$A = 1$$

if:

- no approval is required, or
- approval is required and already granted.

If approval is required but pending:

$$A = 0.$$

If approval has explicitly been denied:

$$\boxed{\text{HARD INFEASIBLE}}.$$

This separates:

"The plan is physically possible"

from:

"The plan is currently authorized to execute."

60 Factor 8: Weather and Obstruction Safety

Define:

$$W_r = \text{weather risk}$$

and:

$$O_r = \text{obstruction risk}$$

with:

$$W_r, O_r \in [0, 1].$$

Then:

$$W = (1 - W_r)(1 - O_r).$$

Example:

$$W_r = 0.10$$

$$O_r = 0.20$$

then:

$$W = 0.9 \times 0.8 = 0.72.$$

If official policy prohibits travel under the current conditions, this weighted factor is bypassed and the plan becomes:

$$\boxed{\text{HARD INFEASIBLE}}.$$

61 Factor 9: Alternative Source Quality

This factor measures the resilience of the plan if the primary source becomes unavailable.

For alternative source j:

$$B_{coverage,j} = \min\left(1, \frac{Q_{\text{transferable},j}}{Q_{\text{required}}}\right).$$

Define:

$$\Delta t_j = \max\left(0, ETA_j - ETA_{\text{primary}}\right).$$

Delay quality is:

$$B_{delay,j} = e^{-\Delta t_j/\tau_B}$$

with prototype:

$$\tau_B = 60 \text{ min}.$$

Then:

$$B_j = B_{coverage,j} \times R_j \times B_{delay,j}$$

where:

$$R_j = \text{reliability of alternative source } j.$$

The system takes:

$$B = \max_j(B_j).$$

If no alternative exists:

$$B = 0.$$

This factor receives a relatively small weight because lack of a backup does not automatically make an otherwise executable primary plan infeasible.

62 Factor 10: Data Confidence

Feasibility calculations depend on the quality of the information being used.

The Data Confidence factor uses:

1. completeness,
2. freshness,
3. source reliability.

Define:

$$M = \text{Data Completeness}$$

$$R_s = \text{Source Reliability.}$$

Freshness is:

$$F_d = e^{-a_d/\tau_d}$$

where:

- a_d = age of decision data,
- τ_d = freshness decay constant.

Prototype:

$$\tau_d = 60 \text{ min.}$$

Then:

$$Q = 0.40M + 0.30F_d + 0.30R_s.$$

Example:

$$M = 0.95$$

$$a_d = 10$$

$$R_s = 0.95.$$

Then:

$$F_d = e^{-10/60} \approx 0.846.$$

Therefore:

$$Q = 0.40(0.95) + 0.30(0.846) + 0.30(0.95)$$

$$Q \approx 0.919.$$

If essential data are completely missing, the system should not merely reduce Q. Instead:

$$\boxed{\texttt{NEEDS_DATA}}$$

for decisions requiring that information.

63 Complete Feasibility Equation

For a candidate that passes all hard constraints:

$$\boxed{F = 1 + 9\Big[0.16R + 0.17E + 0.11V + 0.13S + 0.09D + 0.07C + 0.05A + 0.06W + 0.04B + 0.12Q\Big]}$$

where:

$$R = \text{route feasibility}$$

$$E = \text{ETA margin}$$

$$V = \text{vehicle feasibility}$$

$$S = \text{resource availability}$$

$$D = \text{road information reliability}$$

$$C = \text{mission complexity}$$

$$A = \text{approval readiness}$$

$$W = \text{weather and obstruction safety}$$

$$B = \text{alternative-source quality}$$

$$Q = \text{data confidence.}$$

64 Feasibility Status and Categories

Status is evaluated first:

Status	**Score**	**Meaning**
`INVALID_INPUT`	null	Structural validation failed
`NEEDS_DATA`	null	Essential execution data missing
`HARD_INFEASIBLE`	null	Confirmed hard constraint failed
`SCOREABLE`	1.0–10.0	Weighted score may be calculated

For `SCOREABLE` plans, display bands are:

Score	Interpretation
1.0–3.9	Very Low
4.0–5.9	Low / Questionable
6.0–7.4	Moderate
7.5–8.9	High
9.0–10.0	Very High

Internally, the continuous score should be retained for deterministic ordering. The interface should show the band and at most one decimal place.

For example:

$$F_{internal} = 8.8421$$

should not immediately be reduced to:

High (8.8/10).

65 Worked Example: Direct Delivery

Consider a plan:

- Required water: 300 L
- ETA: 40 min
- Time To Failure: 120 min
- Van capacity: 500 units
- Van available immediately
- Warehouse stock: 1000 L
- Warehouse reserve: 200 L
- Already committed: 100 L
- Route: two open road segments
- Weather risk: 0.05
- Obstruction risk: 0.05
- Data completeness: 0.95
- Data age: 10 min
- Data-source reliability: 0.95

Assume:

$$R = 1.00$$

$$E = 1.00$$

$$V = 1.00$$

$$S = 1.00$$

$$D = 0.819$$

$$C = 1.00$$

$$A = 1.00$$

$$W = 0.903$$

$$B = 0.740$$

$$Q = 0.919.$$

Then:

$$\begin{aligned} F_{\text{raw}} = {} & 0.16(1) + 0.17(1) + 0.11(1) + 0.13(1) \\ & + 0.09(0.819) + 0.07(1) + 0.05(1) + 0.06(0.903) \\ & + 0.04(0.740) + 0.12(0.919) \end{aligned}$$

$$F_{\text{raw}} \approx 0.958.$$

Therefore:

$$F = 1 + 9(0.958)$$

$$\boxed{F_{internal} \approx 9.62}$$

The interface displays **Very High (9.6/10)** together with data-confidence and uncertainty warnings. The extra internal precision is used only for reproducible ordering.

66 Worked Example: Road Closure

Suppose the same plan is active, but one required road segment suddenly becomes:

$$\text{BLOCKED.}$$

Even if:

- the vehicle is available,
- supply exists,
- weather is safe,
- data confidence is high,

the route violates a hard constraint.
Therefore:

$$\boxed{F = \varnothing}$$

and:

$$\boxed{\text{Plan Status} = \text{HARD INFEASIBLE}}$$

The system should then trigger:

```
Road closure detected
        |
        v
Identify affected mission
        |
        v
Invalidate existing route
        |
        v
Generate alternate routes
        |
        v
Recalculate ETA
        |
        v
Recalculate feasibility
        |
        v
Recalculate intrinsic urgency only if need state
or evaluation time changed
```

67 Worked Example: Cross-Shelter Transfer

Suppose Shelter B has:

$$Q_B = 500 \text{ L}$$

with:

$$Q_{reserve,B} = 250 \text{ L}.$$

There are:

$$Q_{committed,B} = 50 \text{ L}$$

already assigned.
Therefore:

$$Q_{transferable,B} = 500 - 250 - 50$$

$$Q_{transferable,B} = 200 \text{ L}.$$

Shelter A requires:

$$150 \text{ L}.$$

Since:

$$200 \geq 150,$$

the transfer is physically feasible.
After transfer:

$$Q_{remaining,B} = 500 - 50 - 150$$

$$Q_{remaining,B} = 300 \text{ L}.$$

Since:

$$300 \geq 250,$$

Shelter B remains above its safety reserve.
Therefore the transfer passes the inventory hard constraint.
If Shelter A instead requested:

$$250 \text{ L},$$

then:

$$250 > 200$$

and the plan becomes:

$$\boxed{\text{HARD INFEASIBLE}}$$

because fulfilling Shelter A's request would create unacceptable downstream risk at Shelter B.

68 Pre-Authorized Contingency Plans

DisasterOps may contain contingency plans that have already been authorized by policy or SOP.

However, urgency must never artificially increase feasibility.

Therefore:

"The situation is extremely urgent"

must never imply:

"The proposed plan is highly feasible."

A pre-authorized contingency changes only its authorization state:

$$A = 1.$$

It must not create a numerical floor for total feasibility. Route risk, ETA margin, vehicle readiness, resource headroom, weather, complexity, and data quality retain their calculated values. Otherwise authorization would be mistaken for physical executability.

The plan may be marked `PREAUTHORIZED` only if the policy identifier, policy version, scope, and validity interval are present in the decision record. Expired or out-of-scope authorization must be treated as pending approval.

The AI itself must not have authority to mark a plan as pre-authorized.

That status must originate from configuration, SOP, or a human authority.

69 Missing Data Behaviour

DisasterOps must distinguish between:

1. information that is desirable but non-essential,
2. information that is essential to execution.

If non-essential information is missing:

$$Q \downarrow$$

and the plan may remain feasible.

If essential information is missing, for example:

- unknown resource quantity,
- unknown vehicle capacity,
- no route data,
- unknown destination,

then DisasterOps should return:

```
FEASIBILITY: UNKNOWN / NOT EXECUTABLE YET

Reason:
Essential information missing.

Action:
Request additional information.
```

The system must not fill missing operational values by guessing.

70 Plan Feasibility Algorithm

```
FUNCTION CalculateFeasibility(plan, current_state):

    CHECK structural validity

    CHECK route exists
    CHECK route contains no blocked segment

    CHECK vehicle exists
    CHECK vehicle operational
    CHECK vehicle capacity

    CHECK source exists
    CHECK transferable inventory
    CHECK donor safety reserve

    CHECK travel prohibition

    CHECK approval state

    CHECK hard deadline

    CHECK essential data

    IF structural validation fails:
        RETURN:
            score = null
            status = INVALID_INPUT
            reasons = violated constraints

    IF essential execution data are missing:
        RETURN:
            score = null
            status = NEEDS_DATA
            missing_fields

    IF any physical, policy, or authorization
```

```
   hard constraint fails:
    RETURN:
        score = null
        status = HARD_INFEASIBLE
        reasons = violated constraints

CALCULATE:
    R = route feasibility
    E = ETA margin
    V = vehicle factor
    S = source/resource factor
    D = road reliability
    C = mission complexity
    A = approval readiness
    W = weather/obstruction factor
    B = alternative-source factor
    Q = data confidence

F_raw =
    0.16*R
  + 0.17*E
  + 0.11*V
  + 0.13*S
  + 0.09*D
  + 0.07*C
  + 0.05*A
  + 0.06*W
  + 0.04*B
  + 0.12*Q

F = 1 + 9*F_raw

RETURN:
    score = F
    status = SCOREABLE
    all factors
    explanation
    assumptions
    weak points
```

71 Relationship Between Urgency and Feasibility

Urgency and feasibility must remain separate.

$$U = \text{Need Priority}$$

while:

F = Plan Executability.

Example:

	Urgency	**Feasibility**
Shelter A, Plan 1	9.5	3.2
Shelter A, Plan 2	9.5	8.8
Shelter B, Plan 1	6.0	9.4

This means:

- Shelter A should receive priority,
- but DisasterOps should select Plan 2 rather than Plan 1.

A plan is not selected merely because the target shelter is urgent.

72 Important Limitation

The feasibility equation and numerical coefficients in this specification are prototype values. They are intended to create:

- deterministic behavior,
- explainability,
- repeatable simulation,
- a clear implementation framework.

They are not claimed to be official disaster-response standards.
Before real-world deployment, DisasterOps should perform:

1. expert validation,
2. SOP calibration,
3. historical-case testing,
4. sensitivity analysis,
5. scenario simulation,
6. failure-mode analysis,
7. threshold validation.

73 Next Required Module

The Feasibility Score answers:

> "Can this candidate plan realistically be executed?"

It does not yet answer:

> "Which candidate plans should DisasterOps create in the first place?"

Therefore the next required system module is the:

Candidate Plan Generator

The Candidate Plan Generator should generate possible combinations of:

- supply source,
- quantity,
- donor shelter,
- vehicle,
- route,
- departure time,
- number of trips,
- cross-shelter transfer,
- alternative source,
- alternative route.

The pipeline will then become:

```
Urgent Need Identified
        |
        v
Candidate Plan Generator
        |
        v
Generate Plan A
Generate Plan B
Generate Plan C
Generate Plan D
        |
        v
Hard Constraint Filter
        |
        v
```

```
Feasibility Score
        |
        v
Feasible Candidates
        |
        v
PLAN UTILITY / RANKING SCORE
        |
        v
Best Recommendation
        |
        v
Human Approval
```

Thus the complete decision architecture will eventually contain three separate mathematical concepts:

$$\boxed{\text{Urgency} \rightarrow \text{Feasibility} \rightarrow \text{Plan Utility / Ranking}}$$

- **Urgency:** Which need should be addressed first?
- **Feasibility:** Can this particular plan actually work?
- **Plan Utility / Ranking:** Among the feasible plans, which plan produces the best overall outcome?

74 Candidate Plan Generation Framework

74.1 Purpose

The Candidate Plan Generator is the component responsible for answering:

> "Given an urgent shelter need and the current disaster state, what realistic response plans are available?"

The Candidate Plan Generator does **not** decide which plan is best.

Its responsibility is to create a sufficiently diverse but bounded set of possible plans.

Each generated candidate is subsequently passed through:

```
Urgent Need
     |
     v
Candidate Plan Generation
     |
     v
Hard Constraint Filtering
     |
```

```
     v
Feasibility Score
     |
     v
Utility / Ranking Score
     |
     v
Recommended Plans
     |
     v
Human Approval
```

Therefore:

$$\boxed{\text{Generation} \neq \text{Evaluation} \neq \text{Selection}}$$

The generator should create alternatives.

The Feasibility Module evaluates whether they can actually work.

The Utility Ranking Module later determines which feasible plan produces the best overall result.

75 Candidate Generator Inputs

The Candidate Plan Generator receives four major inputs:

1. Target Need
2. Current Shared Operational State
3. Operational Rules / SOP Constraints
4. External Tool Data

75.1 Target Need

For need n, define:

$$N_n = \{r, Q_{\text{current}}, C_r, Q_{\text{reserve}}, Q_{\text{requested}}, TTF, U, L_d, t_{\text{deadline}}\}$$

where:

- r = resource type,
- Q_{current} = resource currently available at destination,
- C_r = estimated resource consumption rate,
- Q_{reserve} = minimum safety reserve,
- $Q_{\text{requested}}$ = quantity requested,

- TTF = Time To Failure,
- U = previously calculated Urgency Score,
- L_d = destination shelter location,
- t_{deadline} = any explicit operational deadline.

75.2 Shared Operational State

Candidate generation uses the current shared state:

$$X_t = \{S, W, V, P, R, M, A, E\}$$

where:

- S = shelters,
- W = warehouses and supply sources,
- V = vehicles,
- P = available personnel / drivers,
- R = road network and road status,
- M = active missions,
- A = approvals and authorization state,
- E = environmental conditions such as weather.

Candidate generation must always operate on a specific version of this state.

For example:

$$X_t^{(version=184)}$$

This allows DisasterOps to later determine whether a generated plan has become stale because the system state changed.

76 Candidate Plan Object

Every candidate plan should be represented as a structured object.

Conceptually:

```
CandidatePlan:

    candidate_id

    target_need_id
    target_shelter_id
    resource_type
```

```
service_level
planned_quantity

source_id
source_type

vehicle_ids
driver_ids

departure_time

route_legs

total_distance
estimated_arrival_time
estimated_completion_time

traffic_condition

weather_risk
road_risk

source_inventory_before
source_inventory_after

destination_inventory_before
destination_projected_inventory_after

destination_projected_runway_after

required_approvals

assumptions

data_confidence

state_version

generation_timestamp
```

77 Service Level Generation

DisasterOps should not generate only one delivery quantity.

During a disaster, full fulfillment may not always be possible.

Therefore candidate generation should consider multiple service levels.

77.1 Full Fulfillment Quantity

Define:

$$Q_{\text{full}} = Q_{\text{target}} - Q_{\text{current}} - Q_{\text{incoming}}$$

with:

$$Q_{\text{full}} = \max(0, Q_{\text{full}}).$$

77.2 Target Inventory

The desired target inventory may be calculated using a planning horizon:

$$Q_{\text{target}} = Q_{\text{reserve}} + C_r H_{\text{target}}$$

where:

$$H_{\text{target}} = \text{desired operating horizon.}$$

For example:

$$H_{\text{target}} = 6 \text{ hours.}$$

This is a prototype planning parameter and must eventually come from SOP or coordinator policy.

77.3 Minimum Stabilization Quantity

When full fulfillment is impossible, DisasterOps may generate a smaller stabilization candidate.

Define:

$$Q_{\text{stabilize}} = Q_{\text{reserve}} + C_r H_{\text{minimum}} - Q_{\text{current}} - Q_{\text{incoming}}.$$

Then:

$$Q_{\text{stabilize}} = \max(0, Q_{\text{stabilize}}).$$

For example:

$$H_{\text{minimum}} = 2 \text{ hours}$$

may represent a temporary stabilization period.

Again, this horizon is configurable and must eventually follow operational policy.

77.4 Quantity Candidate Set

The generator may therefore create:

$$\mathcal{Q} = \{Q_{\text{full}}, Q_{\text{stabilize}}, Q_{\text{vehicle-limited}}\}.$$

Duplicate or meaningless quantities are removed.

For example:

```
Need = 500 L water

Candidate Quantity A:
500 L
Full restoration

Candidate Quantity B:
250 L
Temporary stabilization

Candidate Quantity C:
200 L
Maximum quantity available in current vehicle
```

Each quantity represents a different candidate plan.

78 Stage 1: Source Discovery

For resource r, DisasterOps searches all known possible supply sources.

Possible source types include:

- central warehouse,
- secondary warehouse,
- emergency supply depot,
- another shelter with surplus,
- approved third-party source,
- temporary logistics hub.

78.1 Transferable Supply

For source s:

$$Q_{\text{transferable},s} = \max\left(0, Q_{\text{onhand},s} - Q_{\text{reserve},s} - Q_{\text{committed},s} - Q_{\text{buffer},s}\right).$$

where:

- $Q_{\text{onhand},s}$ = physical stock,
- $Q_{\text{reserve},s}$ = safety reserve,
- $Q_{\text{committed},s}$ = resources already assigned,
- $Q_{\text{buffer},s}$ = additional operational buffer.

A source is considered a potential candidate if:

$$Q_{\text{transferable},s} > 0.$$

78.2 Cross-Shelter Transfer

Another shelter may act as a donor source.
However:

$$Q_{\text{donor after transfer}} \geq Q_{\text{reserve}} + Q_{\text{buffer}}.$$

DisasterOps must never solve one shortage by creating another critical shortage.
Cross-shelter transfer candidates should therefore store:

- donor shelter current inventory,
- donor consumption rate,
- donor future projected inventory,
- donor Time To Failure before transfer,
- donor Time To Failure after transfer.

This information will later contribute to the Utility Ranking Score.

79 Stage 2: Vehicle Discovery

For each possible source, DisasterOps searches for vehicles capable of supporting the mission.
Vehicle information includes:

- current location,
- operational status,
- available time,
- maximum load,
- fuel or energy level,
- estimated remaining range,
- driver availability,
- vehicle restrictions,
- currently assigned mission.

79.1 Basic Vehicle Filter

Vehicle v is considered only if:

$$Operational_v = 1.$$

The vehicle must also have sufficient usable range:

$$Range_v \geq Distance_{\text{required}} + Range_{\text{reserve}}.$$

If fuel/range data are not modeled in the MVP, this condition may initially be simplified.

79.2 Vehicle Capacity

For a single-trip candidate:

$$K_v \geq Q_{\text{candidate}}$$

must hold.

If:

$$K_v < Q_{\text{candidate}},$$

the single-trip candidate is not generated.

Instead, DisasterOps may generate:

- multi-trip candidate,
- split-delivery candidate,
- smaller stabilization candidate.

80 Stage 3: Personnel Availability

A vehicle should not automatically be considered usable simply because the vehicle exists.

Where relevant, DisasterOps checks:

$$DriverAvailable_v \in \{0, 1\}.$$

Potential requirements include:

- qualified driver,
- loading personnel,
- unloading personnel,

– special handling certification.

For the hackathon MVP, the minimum model may simply track:

```
Driver available?
YES / NO
```

A candidate requiring unavailable personnel should either:

– not be generated,
– or be generated with a pending personnel requirement.

81 Stage 4: Preliminary Geographic Screening

Route calculations are potentially expensive.

Therefore DisasterOps should not immediately request detailed navigation for every possible combination.

Instead, candidate generation should use a two-stage routing strategy.

81.1 Stage 4A: Route Matrix Screening

For potential source-destination combinations, DisasterOps may use the Google Maps Platform Routes API Compute Route Matrix.

The Route Matrix is used to estimate:

– travel distance,
– estimated travel duration,
– traffic-aware duration where available.

This enables bulk comparison of:

$$Source_1 \rightarrow Shelter_A$$

$$Source_2 \rightarrow Shelter_A$$

$$Source_3 \rightarrow Shelter_A$$

without requesting detailed turn-by-turn route information for every option.

81.2 Vehicle-to-Source Travel

Vehicles may not already be located at the selected source.

Therefore total mission travel time must include:

$$VehicleLocation \rightarrow Source$$

and:

$$Source \rightarrow Destination.$$

Define:

$$T_{\text{deadhead}} = T(VehicleLocation \rightarrow Source).$$

The term "deadhead" refers to vehicle travel before collecting the required load.

82 Total Candidate ETA

Candidate ETA should not simply equal Google Maps source-to-destination driving time.

Define:

$$ETA_{\text{total}} = T_{\text{vehicle wait}} + T_{\text{deadhead}} + T_{\text{loading}} + T_{\text{delivery route}} + T_{\text{operational buffer}}.$$

Therefore:

$$\boxed{ETA_{\text{total}} = T_w + T_d + T_l + T_r + T_b}$$

where:

- T_w = time until vehicle becomes available,
- T_d = vehicle-to-source travel time,
- T_l = loading time,
- T_r = source-to-destination route time,
- T_b = configurable uncertainty / operational buffer.

Optional unloading time may also be included when calculating mission completion:

$$T_{\text{completion}} = ETA_{\text{total}} + T_{\text{unloading}}.$$

83 Stage 5: Preliminary Candidate Pruning

Without pruning, candidate combinations may grow rapidly.
If there are:

$$N_S$$

sources,

$$N_V$$

vehicles,

$$N_Q$$

quantity options,
and:

$$N_R$$

route alternatives,
then naive candidate generation may produce approximately:

$$N_{\text{candidates}} = N_S N_V N_Q N_R.$$

For example:

$$10 \times 8 \times 3 \times 4 = 960$$

possible candidates for only one shelter need.
Therefore DisasterOps applies staged pruning.

83.1 Cheap Pre-Filters

Before expensive detailed routing, remove combinations where:

- transferable stock is zero,
- vehicle is non-operational,
- no driver is available,
- source or destination location is unknown,
- vehicle capacity is clearly insufficient for that candidate type,
- source is forbidden by policy,
- approximate ETA is already far beyond a hard deadline.

83.2 Top-K Source Screening

After Route Matrix calculation, keep only the most promising sources.
For example:

$$K_S = 5.$$

Source pre-screening may consider:

- estimated ETA,
- transferable quantity,
- source data confidence,
- whether the source is a warehouse or donor shelter.

This is only a computational pruning mechanism.
It is not the final Utility Ranking Score.

83.3 Top-K Vehicle Screening

For each shortlisted source, retain only:

$$K_V$$

suitable vehicles.
For example:

$$K_V = 4.$$

Possible preliminary factors include:

- vehicle availability time,
- deadhead ETA,
- capacity,
- operational state.

84 Stage 6: Detailed Google Maps Route Generation

For shortlisted source–vehicle combinations, DisasterOps requests detailed routing.
Google Maps Platform Routes API may be used to obtain:

- primary route,
- alternative routes,

- traffic-aware ETA,
- distance,
- route polyline,
- route legs and route metadata.

The candidate generator should request alternative routes whenever operationally useful.

A route candidate may therefore be represented as:

$$R_k = \{Polyline, Distance, ETA, TrafficState\}.$$

84.1 Traffic-Aware Routing

For current disaster response:

$$T_r = T_{\text{traffic-aware}}.$$

The routing system should use current traffic information when available.

This is particularly useful because disasters may cause:

- congestion,
- emergency road restrictions,
- unusually slow travel,
- evacuation traffic.

85 DisasterOps Road State Layer

Google Maps routing must not be treated as the sole source of truth during a disaster.

DisasterOps maintains its own road-state database.

For road segment e:

$$Status_e \in \{OPEN, RESTRICTED, BLOCKED, UNKNOWN\}.$$

Road-state information may originate from:

- shelter reports,
- municipality staff,
- emergency responders,
- transportation authorities,
- trusted external feeds,
- manually verified reports.

85.1 Route Overlay Check

Every detailed route returned by the external map service must be compared with the DisasterOps road-state layer.

Conceptually:

```
Google route received
       |
       v
Extract route polyline
       |
       v
Compare polyline against DisasterOps road state
       |
       +---- intersects BLOCKED road ----> Reject route
       |
       +---- intersects RESTRICTED road --> Keep with penalty
       |
       +---- intersects UNKNOWN road -----> Keep with low confidence
       |
       v
Accept route candidate
```

85.2 Why a Separate Road Layer Is Required

A local road closure may have been reported to the municipality only minutes earlier.

The external routing provider may not yet contain this information.

Therefore:

$$\boxed{GoogleMapsRoute + DisasterOpsLocalRoadState}$$

should be used together.

A route rejected by the local DisasterOps road layer must not be restored merely because the external map provider suggests it.

86 Custom Road Closure Representation

For the MVP, road closures may be represented as:

- blocked road segment IDs,
- coordinate pairs,
- small geographic polygons,
- simulated edges on a road graph.

Example:

```
ROAD_EVENT:

road_id: ROAD_082

status: BLOCKED

reason: debris

reported_by: Shelter B volunteer

timestamp: 14:32

confidence: 0.85
```

When:

$$Status_{\text{ROAD_082}} : OPEN \rightarrow BLOCKED,$$

all active and candidate routes using that road must be identified.

87 Stage 7: Weather Integration

Weather is included during candidate generation because two geographically different routes may have different environmental risks.

DisasterOps may use the Google Maps Platform Weather API or another approved weather source.

87.1 Weather Inputs

Relevant weather information includes:

- precipitation,
- precipitation probability,
- precipitation type,
- wind speed,
- wind gust,
- visibility,
- thunderstorm probability,
- temperature,
- apparent temperature,
- official weather alerts.

87.2 Weather Should Be Checked Along the Route

Weather should not necessarily be checked only at the destination.

For long routes, the generator may sample:

$$L_1, L_2, \ldots, L_m$$

along the candidate route.

For example:

```
Source
 |
Weather Point 1
 |
Weather Point 2
 |
Weather Point 3
 |
Destination
```

Each point is queried at approximately the time the vehicle is expected to reach that location.

87.3 Time-Dependent Weather

For route point i, define:

$$t_i = t_{\text{departure}} + T_{\text{travel to } i}.$$

Weather should therefore be evaluated using:

$$Weather(L_i, t_i).$$

This allows DisasterOps to account for conditions expected during the mission rather than only conditions at generation time.

87.4 Weather Risk Conversion

Weather API output should not directly be interpreted by the LLM as:

> "This seems dangerous."

Instead, weather observations should be converted into deterministic risk values through configurable rules.

Define:

$$W_i \in [0, 1]$$

as weather risk at sample point i.

Possible factors include:

$$W_i = f(Rain, Wind, Visibility, Thunderstorm, Temperature, Alerts).$$

The route weather risk may be:

$$W_{\text{route}} = \max_i(W_i).$$

The maximum is used because one dangerous section may control the operational safety of the entire route.

87.5 Weather Alerts

If an official weather alert produces an explicit travel prohibition under the applicable SOP:

$$TravelAllowed = 0.$$

The candidate becomes hard infeasible.

Otherwise weather risk is passed to the Feasibility Module as a soft risk factor.

88 Stage 8: Candidate Construction Patterns

DisasterOps should intentionally generate several different types of plans.

This prevents the system from considering only:

> "nearest warehouse + nearest vehicle."

88.1 Pattern A: Direct Warehouse Delivery

```
Warehouse
   |
   v
Shelter
```

This is normally the simplest candidate.

88.2 Pattern B: Alternative Warehouse Delivery

```
Alternative Warehouse
        |
        v
      Shelter
```

Useful when:

- primary warehouse lacks stock,
- primary route is damaged,
- another warehouse produces significantly better ETA.

88.3 Pattern C: Cross-Shelter Transfer

```
Shelter B
surplus resource
      |
      v
Shelter A
urgent shortage
```

Generated only when:

$$Q_{\text{transferable,B}} > 0.$$

The donor shelter's future safety must be calculated.

88.4 Pattern D: Delayed Vehicle Candidate

A currently occupied vehicle may still be usable if it becomes available soon enough.

Example:

```
Van 2 currently busy

Available in:
20 min

Travel:
25 min

Total ETA:
45 min

TTF:
120 min
```

This candidate may be valid even though the vehicle is not available immediately.

88.5 Pattern E: Multi-Trip Candidate

If:

$$Q_{\text{required}} > K_v,$$

DisasterOps may generate a multi-trip plan.

Required number of trips:

$$N_{\text{trips}} = \left\lceil \frac{Q_{\text{required}}}{K_v} \right\rceil.$$

Example:

$$Q_{\text{required}} = 450L$$

$$K_v = 200L.$$

Therefore:

$$N_{\text{trips}} = 3.$$

The plan becomes:

```
Trip 1: 200 L
Trip 2: 200 L
Trip 3: 50 L
```

88.6 Multi-Trip Inventory Simulation

The system must verify that the destination does not reach failure before later trips arrive.

For arrival k:

$$Q_{dest}(t_k) = Q_0 - C_r t_k + \sum_{j=1}^{k} q_j.$$

At every critical time:

$$Q_{dest}(t) \geq Q_{\text{critical}}$$

should hold.

If the shelter reaches failure before the first useful delivery arrives, the candidate should be rejected or marked unacceptable.

88.7 Pattern F: Split Delivery Candidate

Two vehicles may divide one delivery.

For example:

$$Q_{\text{need}} = 400L$$

with:

$$Van_1 = 200L$$

and:

$$Van_2 = 200L.$$

Then:

$$Q_{\text{delivered}} = 200 + 200 = 400L.$$

This produces a candidate containing two parallel missions.

88.8 Split Delivery Limitation

For the MVP, DisasterOps should limit split plans to:

$$N_{\text{vehicles}} \leq 2.$$

This prevents excessive combinatorial growth.

88.9 Pattern G: Partial Stabilization Candidate

If no full-delivery plan can arrive in time, DisasterOps may generate:

> "Deliver enough now to prevent immediate failure, then follow with another mission later."

Example:

```
Full requirement:
500 L

No vehicle can deliver 500 L before TTF.

Van 1 can deliver:
180 L in 20 minutes.

This extends projected runway:
```

```
45 min -> 3 hours.

Candidate:
Immediate stabilization delivery = 180 L.
```

This is especially important in scarce-resource conditions.

88.10 Pattern H: Mission Reassignment Candidate

A vehicle assigned to a lower-priority mission may theoretically be reassigned.

Example:

```
Van 3 currently assigned:
Blankets -> Shelter D

New critical event:
Water -> Shelter A
```

The generator may create:

```
Candidate:
Interrupt Mission D
Reassign Van 3 to Shelter A
```

However, this candidate must record:

- original mission affected,
- original shelter urgency,
- delay caused,
- downstream risk,
- required human approval.

Mission reassignment should normally require human authorization.

89 Candidate Generation and Active Missions

Candidate generation must account for resources already committed to active missions.

For vehicle v:

$$AvailableTime_v = CurrentTime + RemainingMissionTime_v.$$

For resource r:

$$AvailableStock_r = OnHand_r - Committed_r.$$

A candidate must never double-allocate:

- the same physical stock,
- the same vehicle,
- the same driver,
- the same exclusive operational resource.

90 Stage 9: SOP and RAG Checks

Before candidate generation is finalized, the system MUST determine which policy scopes apply. If a required rule is not already present in the versioned policy snapshot, the Agent MUST query the RAG knowledge base and obtain a citable source. Failure to retrieve a required policy produces `NEEDS_DATA` or `POLICY_UNAVAILABLE`; it must not silently mean "no restriction."

Examples:

- Can resources be transferred between shelters?
- Which resources require coordinator approval?
- Is a specific vehicle allowed to carry the resource?
- What safety reserve must a donor shelter retain?
- Which weather condition prohibits travel?
- Are certain roads or zones restricted?

RAG provides:

```
Relevant Rule

Source Document

Section

Citation

Confidence
```

Retrieved text is evidence, not executable authority by itself. A deterministic policy adapter MUST convert approved SOP clauses into typed constraints containing a rule ID, source version, effective interval, scope, and action. The LLM MUST NOT turn ambiguous prose directly into a hard constraint without validation or human confirmation.

RAG should not calculate:

- inventory,
- ETA,
- vehicle capacity,

- route geometry,
- urgency,
- feasibility.

These are handled by deterministic tools.

91 Stage 10: Preliminary Candidate Disposition

Before passing candidates to the full Feasibility calculation, obvious outcomes may be classified cheaply.

Candidates are marked `HARD_INFEASIBLE` for confirmed constraint failures such as:

- blocked required road,
- impossible vehicle load,
- insufficient transferable source inventory,
- non-operational vehicle,
- prohibited travel,
- denied authorization.

Candidates are marked `NEEDS_DATA` for missing essentials such as:

- missing or unknown vehicle reference,
- unknown destination coordinates,
- unknown vehicle capacity,
- unknown source inventory.

This avoids wasting computation while preserving the distinction between "known impossible" and "not yet knowable."

92 Stage 11: Candidate Deduplication

Different generation paths may create nearly identical candidates.

Example:

```
Candidate 17:
Warehouse A
Van 1
Route 1
300 L
ETA 32 min

Candidate 29:
```

```
Warehouse A
Van 1
Route 1
300 L
ETA 32 min
```

These should be merged.

Define candidate identity approximately by:

$$Identity(p) = (Source, VehicleSet, Route, Quantity, DepartureWindow).$$

Near-identical candidates should not be shown separately.

93 Stage 12: Candidate Diversity Preservation

Candidate generation should not retain only the fastest plans.

Otherwise many retained candidates may be nearly identical.

The final candidate set should intentionally preserve diversity across:

- different sources,
- different vehicles,
- different routes,
- full versus stabilization delivery,
- warehouse versus cross-shelter source.

Example:

```
Candidate A:
Fastest warehouse route

Candidate B:
Second warehouse but highly reliable route

Candidate C:
Nearby donor shelter

Candidate D:
Partial stabilization using immediately available vehicle

Candidate E:
Alternative route with lower road/weather risk
```

This provides the Utility Ranking Module with genuinely different strategic choices.

94 Maximum Candidate Count

For the prototype, define:

$$N_{\max} = 30$$

candidate plans per active need.

This value is configurable.

A practical range for the MVP may be:

$$10 \leq N_{\max} \leq 50.$$

The goal is to avoid combinatorial explosion while retaining useful alternatives.

95 Candidate Pruning Heuristic

A lightweight generation score may be used only for pruning.

It must not replace the final Utility Score.

For example:

$$G_p = 0.35G_{\text{ETA}} + 0.25G_{\text{coverage}} + 0.15G_{\text{source}} + 0.15G_{\text{vehicle}} + 0.10G_{\text{confidence}}.$$

This score exists only to decide:

> "Which candidates are worth fully evaluating?"

It must never be presented as the final recommended-plan score.

96 Route Candidate Generation Strategy

The routing pipeline should be:

```
All candidate sources
        |
        v
Google Compute Route Matrix
        |
        v
Fast ETA / distance screening
        |
        v
Keep top source-vehicle pairs
        |
```

```
        v
Google Compute Routes
        |
        v
Primary + alternative routes
        |
        v
Retrieve route polyline
        |
        v
Check DisasterOps local road closures
        |
        v
Check traffic
        |
        v
Check weather along route
        |
        v
Construct CandidatePlan objects
```

97 Why Two Routing Stages Are Used

Using detailed route generation for every combination would be inefficient.
The first stage:

Route Matrix

is used as a cheap geographic screening mechanism.
The second stage:

Detailed Routes

is used only for shortlisted combinations.
This reduces:

- API usage,
- latency,
- computational cost,
- unnecessary weather queries.

98 Weather API Efficiency Strategy

Weather should also be queried only after route pre-screening.

The recommended sequence is:

```
100 raw logistical combinations
          |
          v
Route Matrix pruning
          |
          v
20 route candidates
          |
          v
Hard filtering
          |
          v
10 detailed routes
          |
          v
Weather evaluation
```

Weather responses should be cached for nearby geographic cells and short time periods.

This prevents repeated API calls for routes traveling through the same area.

99 Candidate Generation Algorithm

```
FUNCTION GenerateCandidates(need, shared_state):

    # ----------------------------------------------------------
    # 1. Validate target need
    # ----------------------------------------------------------

    IF essential need information missing:
        REQUEST missing information
        RETURN no final candidates


    # ----------------------------------------------------------
    # 2. Retrieve relevant SOP constraints
    # ----------------------------------------------------------

    rules =
        RetrieveRelevantSOP(
            resource,
```

```
        shelter,
        disaster_type
    )


# ------------------------------------------------------------
# 3. Determine candidate delivery quantities
# ------------------------------------------------------------

quantities =
    GenerateQuantityOptions(
        full_requirement,
        stabilization_requirement,
        vehicle_limited_quantities
    )


# ------------------------------------------------------------
# 4. Discover possible resource sources
# ------------------------------------------------------------

sources =
    FindSources(resource)

FOR each source:

    calculate transferable inventory

    IF transferable inventory <= 0:
        remove source


# ------------------------------------------------------------
# 5. Discover possible vehicles
# ------------------------------------------------------------

vehicles =
    FindOperationalVehicles()

REMOVE vehicles with:
    no known location
    non-operational status
    no available driver
    incompatible resource handling


# ------------------------------------------------------------
# 6. Geographic pre-screening
```

```
# ------------------------------------------------------

route_matrix =
    GoogleRouteMatrix(
        vehicle locations,
        sources,
        target shelter
    )

calculate rough:
    vehicle -> source time
    source -> destination time


# ------------------------------------------------------
# 7. Prune clearly poor combinations
# ------------------------------------------------------

retain top-K sources
retain top-K vehicles per source

reject:
    obvious deadline violations
    impossible capacity combinations
    prohibited sources


# ------------------------------------------------------
# 8. Generate basic plan patterns
# ------------------------------------------------------

GENERATE:
    direct warehouse plans
    alternate warehouse plans
    cross-shelter transfers
    delayed vehicle plans
    partial stabilization plans
    multi-trip plans where required
    limited split-delivery plans
    mission-reassignment plans if policy allows


# ------------------------------------------------------
# 9. Detailed routing
# ------------------------------------------------------

FOR shortlisted plan:
```

```
routes =
    GoogleComputeRoutes(
        origin,
        destination,
        traffic-aware routing,
        request alternatives
    )


FOR each returned route:

    obtain:
        ETA
        distance
        polyline
        route metadata


    # ------------------------------------------------
    # 10. Local disaster road overlay
    # ------------------------------------------------

    compare route polyline
    against DisasterOps RoadState

    IF route intersects BLOCKED road:
        reject route

    IF route intersects RESTRICTED road:
        mark restriction

    IF route intersects UNKNOWN road:
        reduce route confidence


    # ------------------------------------------------
    # 11. Weather
    # ------------------------------------------------

    sample weather along route
    at expected travel times

    calculate:
        weather risk
        visibility risk
        wind risk
        precipitation risk
        weather alert status
```

```
        IF SOP prohibits travel:
            reject route


        # ------------------------------------------------
        # 12. Build candidate object
        # ------------------------------------------------

        candidate =
            CandidatePlan(
                source,
                quantity,
                vehicle,
                driver,
                route,
                ETA,
                traffic,
                weather,
                source inventory,
                donor impact,
                approvals,
                assumptions,
                state version
            )

        add candidate


# ------------------------------------------------------
# 13. Generate multi-trip / split candidates
# ------------------------------------------------------

IF full quantity cannot fit one vehicle:

    generate bounded multi-trip candidates

    generate maximum two-vehicle
    split candidates


# ------------------------------------------------------
# 14. Preliminary hard filter
# ------------------------------------------------------

remove obviously impossible candidates
```

```
    # ------------------------------------------------------------
    # 15. Deduplicate
    # ------------------------------------------------------------

    merge equivalent candidates


    # ------------------------------------------------------------
    # 16. Control candidate explosion
    # ------------------------------------------------------------

    calculate lightweight generation score

    retain candidate diversity

    keep maximum N_max candidates


    # ------------------------------------------------------------
    # 17. Output
    # ------------------------------------------------------------

    RETURN candidate set
```

100 Candidate Generation Output Example

For a water shortage at Shelter A:

```
NEED:

Shelter:
A

Resource:
Water

Required:
300 L

Urgency:
8.7 / 10

TTF:
95 min
```

The Candidate Generator may output:

```
------------------------------------------------
Candidate P1

Source:
Central Warehouse

Quantity:
300 L

Vehicle:
Van 1

Route:
Route A

ETA:
34 min

Traffic:
Moderate

Weather:
Low risk

Service:
Full fulfillment
------------------------------------------------


------------------------------------------------
Candidate P2

Source:
Shelter B

Quantity:
180 L

Vehicle:
Van 3

Route:
Route B

ETA:
18 min

Service:
```

```
Immediate stabilization

Donor effect:
Shelter B remains above safety reserve
------------------------------------------------


------------------------------------------------
Candidate P3

Source:
Secondary Warehouse

Quantity:
300 L

Vehicle:
Van 2

Vehicle available:
15 min

Travel ETA after dispatch:
28 min

Total ETA:
43 min

Service:
Full fulfillment
------------------------------------------------


------------------------------------------------
Candidate P4

Source:
Central Warehouse

Quantity:
300 L

Vehicle:
Van 1

Route:
Alternative Route C
```

```
ETA:
46 min

Weather:
Lower flood exposure than Route A

Service:
Full fulfillment
-------------------------------------------------


-------------------------------------------------
Candidate P5

Source:
Central Warehouse

Quantity:
300 L

Vehicles:
Van 4 + Van 5

Delivery:
150 L + 150 L

ETA first:
22 min

ETA second:
29 min

Service:
Split full fulfillment
-------------------------------------------------
```

These candidates are then passed into the Feasibility Module.

101 Candidate Generation During Replanning

Candidate generation is not used only when the first request arrives.

It must also run whenever an active plan becomes invalid or significantly degraded.

Example:

```
Existing plan:
Warehouse -> Shelter A
```

```
Van 1
Route R1

New event:
Road R1 BLOCKED
```

DisasterOps performs:

```
Detect plan affected
      |
      v
Invalidate Route R1
      |
      v
Preserve unaffected system state
      |
      v
Generate replacement candidates
      |
      v
Try:
Alternative route
Alternative source
Alternative vehicle
Cross-shelter transfer
Partial stabilization
      |
      v
Recalculate feasibility
      |
      v
Recalculate utility ranking
      |
      v
Human approval
```

102 Partial Replanning

A change should not necessarily cause all DisasterOps plans to be regenerated.
Define:

$$AffectedMissions(E)$$

as missions dependent on the state changed by event E.
For example:

$$E = \text{Road R1 blocked.}$$

Then only missions whose routes include:

$$R1$$

must immediately be invalidated.

This reduces computational load and prevents unnecessary disruption.

103 Google Maps Integration Contract

Conceptually, the backend should expose:

```
computeRouteMatrix(
    origins,
    destinations,
    departure_time
)

computeDetailedRoutes(
    origin,
    destination,
    departure_time,
    request_alternatives = true
)

checkRouteAgainstLocalClosures(
    route_polyline,
    road_state
)

calculateTrafficETA(
    route
)
```

The external map service supplies geographic routing information.

DisasterOps remains responsible for:

- interpreting disaster-specific road reports,
- applying local closures,
- enforcing emergency restrictions,
- comparing routes with resource and shelter priorities.

104 Weather Integration Contract

Conceptually:

```
getCurrentWeather(
    latitude,
    longitude
)

getHourlyWeather(
    latitude,
    longitude,
    timestamp
)

getWeatherAlerts(
    latitude,
    longitude
)

evaluateRouteWeather(
    route_polyline,
    departure_time
)
```

The Weather API provides raw environmental information.

DisasterOps converts the data into deterministic operational risk using configurable rules.

105 Candidate Generation Tool Contract

The Candidate Generator may expose functions such as:

```
generateCandidates(
    need_id,
    state_version
)

generateQuantityOptions(
    need
)

findEligibleSources(
    resource_type,
    quantity
)
```

```
calculateTransferableStock(
    source_id,
    resource_type
)

findEligibleVehicles(
    source,
    destination
)

findEligiblePersonnel(
    vehicle
)

screenSourceDestinationPairs(
    sources,
    destination
)

getDetailedRoutes(
    source,
    destination
)

checkLocalRoadState(
    route
)

evaluateWeatherForRoute(
    route,
    departure_time
)

generateDirectCandidates()

generateCrossShelterCandidates()

generateMultiTripCandidates()

generateSplitDeliveryCandidates()

generateStabilizationCandidates()

generateMissionReassignmentCandidates()

deduplicateCandidates()
```

```
pruneCandidates()

preserveCandidateDiversity()
```

106 Candidate Generation Safety Principles

The Candidate Generator must follow the following principles.

106.1 Principle 1: Never Invent Resources

The AI may not assume a vehicle, supply source, road, or inventory exists.
All physical resources must originate from the shared state.

106.2 Principle 2: Never Invent Road Safety

An external route result does not prove that the route is safe after a disaster.
Local road-state information must be checked separately.

106.3 Principle 3: Never Invent Missing Quantity

Unknown stock should remain:

$$UNKNOWN$$

rather than being guessed by the LLM.

106.4 Principle 4: Never Hide Partial Fulfillment

A plan delivering:

$$150L$$

for a:

$$300L$$

need must explicitly state:

$$50\%$$

coverage.
It cannot be presented as full fulfillment.

106.5 Principle 5: Never Create Hidden Downstream Risk

Cross-shelter transfers must expose their impact on the donor shelter.

106.6 Principle 6: Candidate Generation Must Remain Explainable

For every candidate, DisasterOps should be able to answer:

- Why was this source considered?
- Why was this vehicle considered?
- Why was this quantity selected?
- Why was this route generated?
- What traffic information was used?
- What weather information was used?
- What local road reports affected the route?
- Which assumptions were required?

107 Candidate Generation MVP Scope

The first prototype should support:

1. one active target need at a time,
2. warehouse delivery,
3. alternate warehouse delivery,
4. cross-shelter transfer,
5. traffic-aware route generation,
6. alternative road routes,
7. local blocked-road overlay,
8. weather-aware route evaluation,
9. one-vehicle direct delivery,
10. limited multi-trip delivery,
11. maximum two-vehicle split delivery,
12. partial stabilization delivery.

107.1 Not Required for Initial MVP

The initial prototype does not need:

- full city-wide global vehicle routing optimization,
- dozens of simultaneous multi-stop routes,
- autonomous airlift scheduling,
- drone fleet control,
- medical evacuation planning,
- autonomous cancellation of critical missions.

These can be future extensions.

108 Candidate Generation Example After Road Closure

Suppose DisasterOps originally generates:

```
P1:

Warehouse A
-> Shelter A

Van 1

300 L water

ETA:
30 min
```

Then:

```
Road event:

Route segment R17 blocked.
```

Candidate P1 is invalidated.

The generator then searches:

```
Option 1:
Same source
Same vehicle
Alternative Google route
```

```
Option 2:
Different warehouse
Same vehicle

Option 3:
Nearby donor shelter
Different vehicle

Option 4:
Immediate stabilization delivery
from nearer source

Option 5:
Vehicle currently serving
lower-priority mission
requires reassignment approval
```

Each new plan receives a new Feasibility Score.

This demonstrates the agentic property:

$$\boxed{\begin{array}{c} Observe \rightarrow Detect\ invalid\ plan \rightarrow Generate\ alternatives \\ \rightarrow Evaluate \rightarrow Recommend \end{array}}$$

109 Relationship to Feasibility and Utility

The three components must remain distinct.

$$\boxed{CandidateGeneration \rightarrow Feasibility \rightarrow Utility}$$

109.1 Candidate Generation

Question:

> "What can we possibly do?"

109.2 Feasibility

Question:

> "Can this particular plan realistically work?"

109.3 Utility Ranking

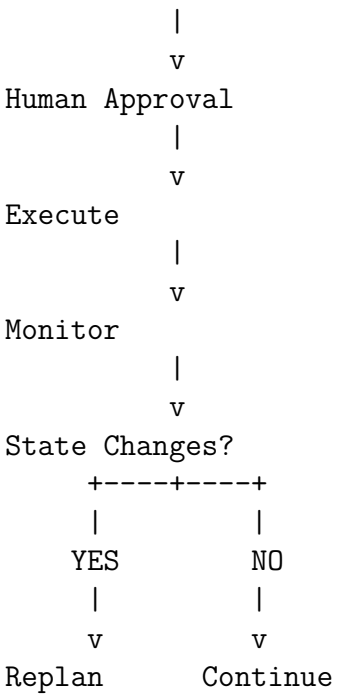
Question:

> "Among the plans that can work, which one gives the best overall outcome?"

110 Complete Planning Pipeline

The DisasterOps decision brain now follows:

```
Shelter Status / New Event
          |
          v
Validate Information
          |
          v
Update Shared State
          |
          v
Calculate Urgency
          |
          v
Select Need for Planning
          |
          v
Candidate Plan Generator
          |
          v
Sources
Vehicles
Quantities
Routes
Traffic
Weather
SOP
          |
          v
Candidate Set
          |
          v
Hard Constraint Filter
          |
          v
Feasibility Score
          |
          v
Feasible Candidate Set
          |
          v
UTILITY RANKING
          |
          v
Top Recommendation
+ Alternatives
```



111 Next Required Module

After Candidate Plan Generation, the next major mathematical module is:

Plan Utility / Ranking Score

The Utility Ranking Module should answer:

> "Several plans are feasible. Which plan produces the best overall system outcome?"

It should consider factors such as:

- target need urgency,
- quantity delivered,
- percentage of need satisfied,
- arrival time,
- predicted shortage avoided,
- feasibility score,
- donor-shelter downstream risk,
- impact on other active missions,
- vehicle usage,
- total travel time,
- total distance,
- resource efficiency,

- weather and route robustness,
- data confidence,
- fairness between shelters,
- opportunity cost.

The Utility Ranking Score must not simply select the shortest ETA.

Its purpose is to evaluate the consequence of each candidate on the **entire operational system**.

112 Plan Utility and Ranking Framework

112.1 Purpose

After candidate generation and feasibility evaluation, DisasterOps may have several plans that are technically executable.

The Utility Ranking Module answers:

> "Among all feasible candidate plans, which plan produces the best overall operational outcome for the disaster-response system?"

The three main decision concepts must remain separate:

Urgency → Candidate Generation → Feasibility → Utility Ranking

They answer different questions.

- **Urgency Score:**

  Which need requires attention first?

- **Candidate Generator:**

  What response plans are possible?

- **Feasibility Score:**

  Can this specific plan realistically be executed?

- **Utility Score:**

  If this plan is executed, how good is its total effect on the entire operational system?

113 Important Design Principle

The Utility Score must not simply be:

$$Urgency + Feasibility.$$

This would double-count many variables.
For example:

- urgency already considers time pressure,
- feasibility already considers ETA,
- feasibility already considers weather,
- feasibility already considers route reliability,
- feasibility already considers vehicle availability.

Therefore Utility Ranking focuses primarily on:

1. reduction of system-wide unmet critical need,
2. how much of the requested need is actually satisfied,
3. downstream harm created elsewhere,
4. disruption of existing missions,
5. operational efficiency,
6. preservation of future response flexibility.

114 Feasibility Gate Before Utility

Utility is only calculated for candidates that have already passed the Feasibility Module.
If:

$$Status_p \in \{\texttt{INVALID_INPUT}, \texttt{NEEDS_DATA}, \texttt{HARD_INFEASIBLE}\},$$

then:

$$\boxed{Plan\ p \text{ is excluded from Utility Ranking}}$$

For the prototype, DisasterOps may define a recommended feasibility threshold:

$$F_{\min} = 6.0.$$

If:

$$F_p < F_{\min},$$

the plan may remain visible as an emergency fallback but should not normally become the primary recommendation unless no stronger alternative exists.
Therefore:

```
Invalid / Needs Data / Hard Infeasible
      |
      v
REMOVE

Feasible but F < 6
      |
      v
FALLBACK ONLY

F >= 6
      |
      v
NORMAL UTILITY RANKING
```

The threshold:

$$F_{\min} = 6.0$$

is a prototype assumption and must later be calibrated.

115 Utility Score Definition

For feasible candidate plan p, define:

$$J_p \in [0, 1]$$

as normalized plan utility.
The internal deterministic score is:

$$\boxed{Utility_p = 1 + 9J_p}$$

Therefore:

$$1 \leq Utility_p \leq 10.$$

The user interface should show the category and at most one decimal place. It must also show the ranking run's confidence and principal uncertainties; internal extra precision exists only for stable sorting and audit reproduction.

116 Prototype Utility Formula

The preliminary DisasterOps utility model is:

$$\boxed{J_p = 0.40I_p + 0.20C_p + 0.15D_p + 0.10M_p + 0.10E_p + 0.05O_p}$$

where:

$$I_p = \text{System Impact}$$

$$C_p = \text{Effective Need Coverage}$$

$$D_p = \text{Downstream Safety}$$

$$M_p = \text{Existing Mission Continuity}$$

$$E_p = \text{Operational Efficiency}$$

$$O_p = \text{Future Operational Optionality.}$$

All factors satisfy:

$$0 \leq I_p, C_p, D_p, M_p, E_p, O_p \leq 1.$$

116.1 Factor Weights

Factor	**Symbol**	**Weight**
System-wide risk reduction	I_p	0.40
Effective need coverage	C_p	0.20
Downstream safety	D_p	0.15
Existing mission continuity	M_p	0.10
Operational efficiency	E_p	0.10
Future operational optionality	O_p	0.05
Total		**1.00**

These weights are prototype engineering assumptions.
They do not represent validated emergency-management doctrine.

117 Factor 1: System Impact

System Impact is the most important utility factor.
It measures:

> "How much does this plan reduce the total unresolved emergency burden across the entire shelter system?"

This is intentionally different from evaluating only the target shelter.

117.1 Normalized Urgency

For active need i, calculate urgency in the common baseline state X_0:

$$U_i^0 = Urgency(i, X_0) \in [1, 10].$$

Normalize it once:

$$u_i^0 = \frac{U_i^0 - 1}{9}.$$

Therefore:

$$u_i^0 \in [0, 1].$$

The active-need cohort and these urgency weights are frozen for all candidates in the same ranking run. Recomputing different weights or denominators for different candidates would make their burden values non-comparable.

117.2 Normalized Unmet Need

Define:

$$Q_{\text{target},i} = \text{desired resource quantity for need } i.$$

Let:

$$Q_{\text{projected},i}(X)$$

represent the projected available quantity under system state X.
Then the normalized unmet need is:

$$m_i(X) = \text{clamp}\left(\frac{Q_{\text{target},i} - Q_{\text{projected},i}(X)}{Q_{\text{target},i}}, 0, 1\right).$$

where:

$$\text{clamp}(x, 0, 1) = \max(0, \min(1, x)).$$

117.3 Need Burden

The burden associated with need i is:

$$b_i(X) = u_i^0 m_i(X).$$

A highly urgent need with a large shortage creates high burden.
A minor shortage with low urgency creates smaller burden.

117.4 Total System Burden

For the fixed active-need cohort captured from X_0:

$$\mathcal{N} = \{1, 2, \ldots, n\},$$

define:

$$B(X) = \frac{\sum_{i=1}^n u_i^0 m_i(X)}{\sum_{i=1}^n u_i^0 + \epsilon}$$

where:

$$\epsilon$$

is a small constant preventing division by zero.
For implementation:

$$\epsilon = 10^{-6}$$

is sufficient.
Therefore:

$$0 \leq B(X) \leq 1.$$

117.5 Before and After Simulation

Let:

$$X_0$$

be the projected system state without executing candidate plan p.
Let:

$$X_p$$

be the projected state after executing candidate plan p.
Then:

$$B_0 = B(X_0)$$

and:

$$B_p = B(X_p).$$

System Impact is:

$$\boxed{I_p = \text{clamp}\left(\frac{B_0 - B_p}{B_0 + \epsilon}, 0, 1\right)}$$

Interpretation:

$$I_p = 1$$

means the plan removes essentially all relevant projected burden.

$$I_p = 0$$

means the plan produces no overall improvement.

117.6 Why System Impact Is Important

Consider:

```
Shelter A:
Urgent water shortage

Shelter B:
Moderate water reserve
```

Candidate 1:

```
Take almost all spare water from B
and deliver it to A.
```

Shelter A improves significantly.
However Shelter B may become vulnerable.
Therefore:

$$B_A \downarrow$$

but:

$$B_B \uparrow .$$

System Impact automatically captures both changes.
This prevents DisasterOps from optimizing one shelter while ignoring the rest of the system.

118 Evaluation Horizon

System states must be compared at a common future time.
Define:

$$H_{\text{eval}} = \text{Utility Evaluation Horizon.}$$

Prototype examples:

$$H_{\text{eval}} = 2 \text{ hours}$$

or:

$$H_{\text{eval}} = 4 \text{ hours.}$$

The exact value should depend on operational policy and scenario duration.
All candidate plans should be projected to the same:

$$t + H_{\text{eval}}.$$

The state simulation should include:

- resource consumption,

- incoming deliveries,
- donor transfers,
- vehicle missions,
- mission delays,
- population changes if known,
- road and weather assumptions where relevant.

119 Factor 2: Effective Need Coverage

A plan should receive credit according to how much useful supply actually reaches the target.

Simply planning:

$$300L$$

does not mean the need is fully satisfied if the delivery arrives too late.

119.1 Multi-Delivery Candidate

Suppose candidate p contains deliveries:

$$k = 1, 2, \ldots, K.$$

Each delivery provides:

$$q_k$$

at arrival time, measured in minutes from the ranking run's common `as_of` timestamp:

$$t_k.$$

TTF must use the same unit and the same `as_of` timestamp. Absolute clock timestamps must be converted before this formula is evaluated.

119.2 Late Delivery Discount

Let:

$$TTF = \text{Time To Failure.}$$

Define lateness:

$$L_k = \max\left(0, t_k - TTF\right).$$

Define a lateness discount:

$$d_k = e^{-L_k/\tau_L}.$$

where:

$$\tau_L$$

is a configurable lateness sensitivity parameter.
Prototype:

$$\tau_L = 60 \text{ minutes}.$$

119.3 Interpretation

If:

$$t_k \leq TTF,$$

then:

$$L_k = 0$$

and:

$$d_k = 1.$$

Therefore the delivery receives full value.
If the delivery arrives after TTF:

$$d_k < 1.$$

The later it arrives, the less useful it becomes.

119.4 Effective Delivered Quantity

Define the raw slack:

$$Q_{\text{effective}} = \sum_{k=1}^{K} q_k d_k.$$

Then:

$$\boxed{C_p = \min\left(1, \frac{Q_{\text{effective}}}{Q_{\text{required}}}\right)}$$

where:

$$Q_{\text{required}}$$

is the amount required to reach the current planning target.

119.5 Example

Suppose:

$$Q_{\text{required}} = 300L.$$

Candidate delivers:

$$200L$$

before TTF.
Then:

$$C_p = \frac{200}{300} = 0.667.$$

A partial stabilization plan therefore receives meaningful credit without being treated as full fulfillment.

120 Factor 3: Downstream Safety

Downstream Safety measures whether the candidate creates new risk elsewhere.

This is particularly important for:

- cross-shelter transfers,
- warehouse depletion,
- scarce-resource reallocation.

120.1 Remaining Buffer

For affected source s:

$$Q_{\text{after},s} = \text{projected inventory after candidate execution.}$$

Define:

$$Q_{\text{reserve},s} = \text{required safety reserve.}$$

Define:

$$Q_{\text{buffer},s} = \text{desired additional operational buffer.}$$

Then:

$$D_s = \text{clamp}\left(\frac{Q_{\text{after},s} - Q_{\text{reserve},s}}{Q_{\text{buffer},s}}, 0, 1\right).$$

120.2 Plan Downstream Safety

For all affected sources:

$$\mathcal{S}_p,$$

define:

$$\boxed{D_p = \min_{s \in \mathcal{S}_p} D_s}$$

The minimum is used because the weakest affected source determines downstream safety.

If no donor shelter or strategic source is meaningfully depleted:

$$D_p = 1.$$

120.3 Hard Safety Rule

If:

$$Q_{\text{after},s} < Q_{\text{reserve},s},$$

the plan should already have been rejected by the Feasibility Module.

Utility must never compensate for violating a safety reserve.

121 Factor 4: Existing Mission Continuity

Some candidate plans may require:

- reassigning a vehicle,
- delaying another delivery,
- cancelling a lower-priority mission,
- consuming a shared resource already expected elsewhere.

These effects must be explicitly measured.

121.1 Mission Delay

For affected existing mission j, define:

$$ETA_j^{base} = \text{original expected arrival time}$$

and:

$$ETA_j^{p} = \text{arrival time after candidate } p.$$

Additional delay is:

$$\Delta t_j = \max\left(0, ETA_j^{p} - ETA_j^{base}\right).$$

121.2 Mission Slack

Define:

$$Slack_j^{raw} = TTF_j - ETA_j^{base}.$$

To prevent division by zero:

$$Slack_j^{+} = \max(Slack_j^{raw}, \epsilon_t)$$

where:

$$\epsilon_t = 1 \text{ minute}.$$

121.3 Disruption Penalty

Define:

$$r_j = \min\left(1, \frac{\Delta t_j}{Slack_j^{+}}\right).$$

Normalize urgency:

$$u_j = \frac{U_j - 1}{9}.$$

Then Mission Continuity is:

$$\boxed{M_p = 1 - \frac{\sum_j u_j r_j}{\sum_j u_j + \epsilon}}$$

If no existing missions are disrupted:

$$M_p = 1.$$

121.4 Critical Mission Override

If candidate p causes an existing critical mission to cross a hard operational deadline:

$$ETA_j^p > Deadline_j,$$

the candidate should be rejected or require explicit emergency override.
Similarly, if mission reassignment is predicted to create:

$$U_j \geq 9,$$

for another shelter, the candidate should be flagged:

```
SEVERE DOWNSTREAM IMPACT
HUMAN OVERRIDE REQUIRED
```

122 Factor 5: Operational Efficiency

Among plans with similar emergency outcomes, DisasterOps should prefer plans that use fewer scarce logistical resources.

Efficiency should remain lower priority than safety and impact.

122.1 Vehicle-Time Cost

Define:

$$T_{vehicle,p} = \text{total vehicle-minutes consumed by candidate } p.$$

Normalize:

$$e_T = \text{clamp}\left(\frac{T_{vehicle,p}}{T_{\text{ref}}}, 0, 1\right).$$

Prototype example:

$$T_{\text{ref}} = 180 \text{ vehicle-minutes}.$$

122.2 Distance Cost

Define:

$$D_{km,p} = \text{total vehicle distance}.$$

Normalize:

$$e_D = \text{clamp}\left(\frac{D_{km,p}}{D_{\text{ref}}}, 0, 1\right).$$

Prototype:

$$D_{\text{ref}} = 100 \text{ km}.$$

122.3 Trip Cost

Define:

$$N_{trip,p} = \text{number of trips}.$$

Then:

$$e_N = \text{clamp}\left(\frac{N_{trip,p} - 1}{N_{\text{ref}} - 1}, 0, 1\right).$$

Prototype:

$$N_{\text{ref}} = 4.$$

122.4 Load Utilization

For vehicle trip k:

$$L_k = \frac{Load_k}{Capacity_k}.$$

Average load utilization is:

$$\bar{L} = \frac{1}{K} \sum_{k=1}^{K} L_k.$$

Higher utilization generally indicates more efficient use of vehicles.

122.5 Efficiency Score

Define:

$$\boxed{E_p = 1 - \left[0.45e_T + 0.25e_D + 0.15e_N + 0.15(1 - \bar{L})\right]}$$

Then clamp:

$$E_p = \text{clamp}(E_p, 0, 1).$$

Operational efficiency must never override safety.
Its purpose is mainly to distinguish otherwise similarly effective plans.

123 Factor 6: Future Operational Optionality

A good plan should avoid unnecessarily consuming every available asset.
Maintaining spare capacity gives DisasterOps flexibility when new emergencies appear.

123.1 Vehicle Reserve

Define:

$$V_{\text{operational}} = \text{number of operational vehicles.}$$

Let:

$V_{\text{available-after}}$ = vehicles expected to remain available after candidate commitment.

Then:

$$O_V = \frac{V_{\text{available-after}}}{V_{\text{operational}} + \epsilon}.$$

123.2 Resource Reserve

For strategic resource types r:

$$Surplus_r = \max\left(0, Q_r - Q_{\text{reserve},r}\right).$$

Define a target system buffer:

$$BufferTarget_r.$$

Then:

$$O_R = \frac{1}{|\mathcal{R}|} \sum_{r \in \mathcal{R}} \text{clamp}\left(\frac{Surplus_r}{BufferTarget_r}, 0, 1\right).$$

123.3 Optionality Score

Then:

$$\boxed{O_p = 0.50O_V + 0.50O_R}$$

A plan that consumes every remaining vehicle and almost all emergency inventory receives lower optionality.

This factor receives only 5% weight because immediate emergency impact should dominate future flexibility.

124 Complete Utility Equation

For a feasible candidate plan p:

$$\boxed{J_p = 0.40I_p + 0.20C_p + 0.15D_p + 0.10M_p + 0.10E_p + 0.05O_p}$$

The final Utility Score is:

$$\boxed{Utility_p = 1 + 9J_p}$$

125 Utility Categories

Utility Score	Interpretation
1.0–2.9	Very Poor System Outcome
3.0–4.9	Poor
5.0–6.4	Acceptable / Limited Benefit
6.5–7.9	Good
8.0–8.9	Very Good
9.0–10.0	Excellent Overall Outcome

126 Worked Example

Suppose Shelter A has:

$$U_A = 9.0$$

and requires:

$$300L$$

of water.
Three feasible candidates remain.

126.1 Candidate P1: Central Warehouse

```
Source:
Central Warehouse

Delivery:
300 L

ETA:
35 min

No other mission disturbed

No donor shelter affected

Moderate travel distance

Feasibility:
9.3 / 10
```

Assume:

$$I_1 = 0.82$$

$$C_1 = 1.00$$

$$D_1 = 1.00$$

$$M_1 = 1.00$$

$$E_1 = 0.75$$

$$O_1 = 0.70.$$

Then:

$$J_1 = 0.40(0.82) + 0.20(1) + 0.15(1) + 0.10(1) + 0.10(0.75) + 0.05(0.70)$$

$$J_1 = 0.328 + 0.200 + 0.150 + 0.100 + 0.075 + 0.035$$

$$J_1 = 0.888.$$

Therefore:

$$Utility_1 = 1 + 9(0.888)$$

$$\boxed{Utility_1 \approx 9.0/10}$$

126.2 Candidate P2: Nearby Shelter Transfer

```
Source:
Shelter B

Delivery:
180 L

ETA:
18 min

Very fast

Partial stabilization only

Shelter B loses much of its spare buffer

Feasibility:
8.8 / 10
```

Assume:

$$I_2 = 0.62$$

$$C_2 = 0.60$$

$$D_2 = 0.55$$

$$M_2 = 1.00$$

$$E_2 = 0.92$$

$$O_2 = 0.60.$$

Then:

$$J_2 = 0.40(0.62) + 0.20(0.60) + 0.15(0.55) + 0.10(1) + 0.10(0.92) + 0.05(0.60)$$

$$J_2 = 0.248 + 0.120 + 0.0825 + 0.100 + 0.092 + 0.030$$

$$J_2 = 0.6725.$$

Therefore:

$$Utility_2 = 1 + 9(0.6725)$$

$$\boxed{Utility_2 \approx 7.1/10}$$

Candidate P2 is much faster than P1.
However, it:

- satisfies only part of the requirement,
- reduces Shelter B's safety margin,
- leaves less system reserve.

Therefore it receives lower total utility.

126.3 Candidate P3: Vehicle Reassignment

```
Source:
Central Warehouse

Delivery:
300 L

ETA:
25 min

Requires Van 3 to abandon
a lower-priority delivery

Feasibility:
8.6 / 10
```

Assume:

$$I_3 = 0.70$$

$$C_3 = 1.00$$

$$D_3 = 1.00$$

$$M_3 = 0.35$$

$$E_3 = 0.80$$

$$O_3 = 0.40.$$

Then:

$$J_3 = 0.40(0.70) + 0.20(1) + 0.15(1) + 0.10(0.35) + 0.10(0.80) + 0.05(0.40)$$

$$J_3 = 0.280 + 0.200 + 0.150 + 0.035 + 0.080 + 0.020$$

$$J_3 = 0.765.$$

Therefore:

$$Utility_3 = 1 + 9(0.765)$$

$$\boxed{Utility_3 \approx 7.9/10}$$

127 Final Ranking Example

The candidates are ranked:

Candidate	**Feasibility**	**Utility**
P1: Warehouse Direct	9.3	9.0
P3: Vehicle Reassignment	8.6	7.9
P2: Cross-Shelter Transfer	8.8	7.1

DisasterOps recommends:

$$\boxed{P1}$$

even though P1 is not the fastest plan.
This demonstrates an important design principle:

> The fastest plan is not necessarily the best system-wide plan.

128 Pareto Dominance Filter

Before weighted utility ranking, DisasterOps may remove clearly dominated candidates.

Candidate p dominates candidate q if:

$$I_p \geq I_q$$

$$C_p \geq C_q$$

$$D_p \geq D_q$$

$$M_p \geq M_q$$

$$E_p \geq E_q$$

$$O_p \geq O_q$$

and at least one inequality is strict.

Then:

$$p \succ q.$$

Candidate q is Pareto dominated and may be removed from the final recommendation list.

This prevents DisasterOps from presenting obviously inferior alternatives.

129 Utility Ranking Algorithm

```
FUNCTION RankPlans(
    feasible_candidates,
    current_state
):

    # ------------------------------------------------------
    # 1. Remove invalid plans
    # ------------------------------------------------------

    REMOVE candidates marked HARD_INFEASIBLE


    # ------------------------------------------------------
    # 2. Separate fallback plans
    # ------------------------------------------------------

    normal_candidates = []
    fallback_candidates = []
```

```
FOR candidate:

    IF feasibility >= F_min:
        normal_candidates.append(candidate)

    ELSE:
        fallback_candidates.append(candidate)


# ------------------------------------------------------
# 3. Simulate each candidate
# ------------------------------------------------------

FOR candidate p:

    projected_state =
        SimulatePlan(
            current_state,
            candidate,
            evaluation_horizon
        )


    # ------------------------------------------------------
    # 4. Calculate system-wide impact
    # ------------------------------------------------------

    B_before =
        CalculateSystemBurden(
            current_state
        )

    B_after =
        CalculateSystemBurden(
            projected_state
        )

    I_p =
        clamp(
            (B_before - B_after)
            /
            (B_before + epsilon),
            0,
            1
        )


    # ------------------------------------------------------
```

```
# 5. Calculate effective need coverage
# ------------------------------------------------------

C_p =
    CalculateEffectiveCoverage(
        deliveries,
        required_quantity,
        TTF
    )


# ------------------------------------------------------
# 6. Calculate downstream safety
# ------------------------------------------------------

D_p =
    CalculateDownstreamSafety(
        affected_sources,
        projected_state
    )


# ------------------------------------------------------
# 7. Calculate mission continuity
# ------------------------------------------------------

M_p =
    CalculateMissionContinuity(
        affected_existing_missions
    )


# ------------------------------------------------------
# 8. Calculate operational efficiency
# ------------------------------------------------------

E_p =
    CalculateOperationalEfficiency(
        vehicle_minutes,
        distance,
        trips,
        load_utilization
    )


# ------------------------------------------------------
# 9. Calculate future optionality
# ------------------------------------------------------
```

```
    O_p =
        CalculateOperationalOptionality(
            remaining_vehicles,
            remaining_resource_buffers
        )


    # ------------------------------------------------------
    # 10. Calculate normalized utility
    # ------------------------------------------------------

    J_p =
          0.40 * I_p
        + 0.20 * C_p
        + 0.15 * D_p
        + 0.10 * M_p
        + 0.10 * E_p
        + 0.05 * O_p


    Utility_p =
        1 + 9 * J_p


    STORE:
        Utility_p
        all component factors
        projected consequences


# ----------------------------------------------------------
# 11. Remove Pareto dominated plans
# ----------------------------------------------------------

remove dominated candidates


# ----------------------------------------------------------
# 12. Sort
# ----------------------------------------------------------

SORT candidates
by Utility descending


# ----------------------------------------------------------
# 13. Return recommendation
```

```
    # ------------------------------------------------------

    RETURN:
        recommended_plan
        second_best_plan
        third_best_plan
        explanations
```

130 Tie-Breaking Rules

If two plans have very similar Utility Scores:

$$|Utility_a - Utility_b| < 0.2,$$

DisasterOps should use deterministic tie-breakers.
Recommended order:

1. Higher System Impact I
2. Higher Downstream Safety D
3. Higher Feasibility Score F
4. Higher Mission Continuity M
5. Lower ETA
6. Lower vehicle-resource consumption

131 No Feasible High-Utility Candidate

If all candidates are:

- hard infeasible,
- low feasibility,
- or extremely low utility,

DisasterOps must not invent a solution.
Instead:

```
NO ACCEPTABLE PLAN FOUND

Reason:
Current resources / roads / vehicles
cannot satisfy the need safely.

Recommended action:
Escalate to municipal coordinator.
```

```
Possible actions:
- request external resources
- request additional vehicles
- request emergency transport
- verify alternate road
- reassess lower-priority missions
```

132 Road Closure Re-Ranking

Suppose Plan P1 is originally selected.
Then a road closure occurs.
The route becomes hard infeasible.
DisasterOps performs:

```
Road closure detected
        |
        v
Identify affected plan
        |
        v
Set current plan:
HARD INFEASIBLE
        |
        v
Update Shared State
        |
        v
Generate replacement candidates
        |
        v
Google Maps:
New route alternatives
        |
        v
Weather:
Evaluate new routes
        |
        v
Feasibility recalculation
        |
        v
State simulation
        |
        v
Utility recalculation
        |
        v
Re-rank candidates
```

```
        |
        v
New recommendation
        |
        v
Human approval
```

133 Google Maps and Weather in Utility Ranking

Google Maps and Weather data primarily influence:

- candidate generation,
- ETA,
- feasibility,
- projected state timing.

They should not be inserted again as large independent Utility factors because doing so would double-count their effect.

For example:

```
Heavy rain
     |
     v
Route travel slower
     |
     v
ETA increases
     |
     v
Feasibility decreases
     |
     v
Projected delivery occurs later
     |
     v
Effective Coverage decreases
     |
     v
System Impact decreases
```

Thus weather naturally propagates through the decision system.

134 Avoiding Double Counting

The following variables are intentionally NOT directly added to the Utility formula:

- Urgency Score,
- Feasibility Score,
- raw ETA,
- raw weather severity,
- raw traffic congestion,
- raw route reliability.

Instead:

- urgency weights system burden,
- feasibility acts as an execution gate,
- ETA affects effective delivery timing,
- weather affects feasibility and ETA,
- traffic affects ETA,
- route reliability affects feasibility.

This keeps the architecture mathematically cleaner.

135 Human-Readable Recommendation Output

The AI should never return only:

> "Plan P1 = 9.0/10"

Instead it should return:

```
RECOMMENDED PLAN: P1

Utility:
9.0 / 10

Feasibility:
9.3 / 10

Target:
Shelter A

Need:
300 L drinking water

Source:
Central Warehouse
```

```
Vehicle:
Van 1

ETA:
35 min

WHY THIS PLAN WAS SELECTED:

+ Fully satisfies the water requirement
+ Arrives before projected shortage
+ Does not reduce another shelter's reserve
+ Does not disrupt existing critical missions
+ Keeps other vehicles available
+ Produces the largest projected reduction
  in system-wide unmet need

ALTERNATIVE P3:

Utility:
7.9 / 10

Reason not selected:
Requires delaying an existing delivery.

ALTERNATIVE P2:

Utility:
7.1 / 10

Reason not selected:
Faster, but only partially satisfies the need
and significantly reduces Shelter B's reserve.
```

136 Utility Explanation Object

For implementation, every ranked plan should store:

```
UtilityExplanation:

    candidate_id

    utility_score

    feasibility_score

    system_impact
    need_coverage
```

```
downstream_safety
mission_continuity
operational_efficiency
future_optionality

projected_state_version

reasons_for_selection

assumptions

affected_missions

affected_shelters

rejected_alternatives

ranking_timestamp
```

137 Ranking Output

The system should normally display:

1. Recommended Plan
2. Alternative Plan 1
3. Alternative Plan 2

The coordinator should not normally be shown dozens of plans.
The complete candidate set may remain available in technical logs.

138 Human Approval

The final recommendation remains a recommendation.
The coordinator may:

- approve,
- modify,
- reject,
- select another candidate,
- request new candidates.

Human decisions become new system events.
For example:

$$HumanReject(P_1)$$

causes:

$$State_t \rightarrow State_{t+1}.$$

The ranking system may then recommend:

$$P_2.$$

139 Simultaneous Critical Needs

The MVP may process the highest-priority need first.
However, a future version may optimize several critical needs simultaneously.
Instead of selecting one plan:

$$p^*,$$

the system would select a set of plans:

$$\mathcal{P}^*$$

subject to shared constraints:

$$\text{vehicle capacity},$$

$$\text{resource availability},$$

$$\text{road constraints},$$

$$\text{mission timing}.$$

This becomes a larger multi-resource optimization problem.
Possible future approaches include:

- Mixed Integer Linear Programming,
- constraint programming,
- vehicle routing optimization,
- rolling-horizon optimization.

This is not required for the initial hackathon MVP.

140 Prototype Simplification

For the hackathon MVP, DisasterOps may rank plans independently for the currently selected high-priority need.

The practical pipeline is:

```
Calculate urgency for all needs

        |
        v

Select highest-priority need

        |
        v

Generate 10--30 candidate plans

        |
        v

Remove hard-infeasible plans

        |
        v

Calculate Feasibility

        |
        v

Simulate feasible plans

        |
        v

Calculate Utility

        |
        v

Return Top 3

        |
        v

Human Approval
```

141 Complete DisasterOps Decision Brain

The core DisasterOps decision architecture is therefore:

```
============================================================

                  DISASTEROPS

============================================================

INPUT

Shelter reports
Inventory
Population
Vulnerable groups
Vehicles
Drivers
Road conditions
Traffic
Weather
SOPs
Active missions

                      |
                      v

              DATA VALIDATION

Missing information?
Duplicate?
Conflict?
Stale data?

                      |
                      v

              SHARED STATE UPDATE

                      |
                      v

               URGENCY ENGINE

Question:
What needs attention first?

Output:
Status + Urgency 1--10 or null
```

```
                    |
                    v

          CANDIDATE PLAN GENERATOR

Question:
What could we do?

Generate:
Sources
Quantities
Vehicles
Routes
Alternatives
Cross-shelter transfers
Partial deliveries
Multi-trip plans

Use:
Google Maps
Traffic data
Weather API
Local road-state data
RAG / SOP rules

                    |
                    v

          HARD CONSTRAINT FILTER

Remove:
Blocked routes
Insufficient stock
Invalid vehicle
Unsafe donor transfer
Denied authorization
Impossible deadline

                    |
                    v

            FEASIBILITY ENGINE

Question:
Can this plan realistically work?

Output:
```

```
Status + Feasibility 1--10 or null

                      |
                      v

             STATE SIMULATION

Predict:
Inventory after plan
Shelter conditions
Mission delays
Vehicle availability
Future shortages

                      |
                      v

             UTILITY ENGINE

Question:
Which feasible plan produces
the best overall system outcome?

Evaluate:
System impact
Need coverage
Downstream safety
Mission continuity
Efficiency
Future optionality

Output:
Utility 1--10

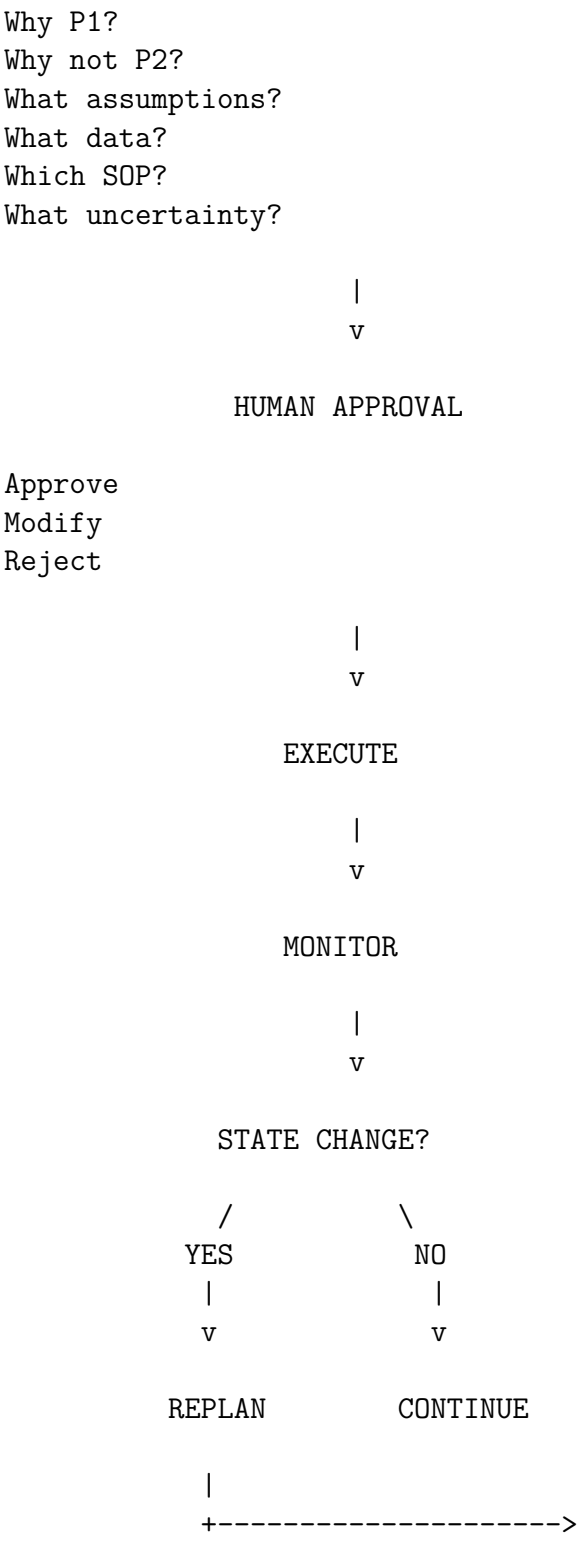
                      |
                      v

               PLAN RANKING

P1
P2
P3

                      |
                      v

              EXPLAIN DECISION
```



==========

142 Core Mathematical Architecture Summary

The primary mathematical sequence is:

$$\boxed{U_i = Urgency(i, X_t, t, \theta_v)}$$

where θ_v is the versioned configuration snapshot. Urgency is intrinsic to the need and does not consume candidate ETA.

$$\boxed{\mathcal{P}_i = GenerateCandidates(i, X_t, \theta_v)}$$

For every:

$$p \in \mathcal{P}_i,$$

calculate:

$$\boxed{F_p = Feasibility(p, X_t, \theta_v)}$$

Only candidates whose Feasibility status is `SCOREABLE` continue.
Then simulate:

$$\boxed{X_t \xrightarrow{p} X_{t+H}}$$

Every candidate in the ranking run uses the same baseline X_t, fixed active-need cohort, urgency weights, evaluation horizon H, and configuration θ_v.
and calculate:

$$\boxed{J_p = 0.40I_p + 0.20C_p + 0.15D_p + 0.10M_p + 0.10E_p + 0.05O_p}$$

Then:

$$\boxed{Utility_p = 1 + 9J_p}$$

Finally:

$$\boxed{p^* = \arg\max_p Utility_p}$$

subject to:

$$Status_F(p) = \texttt{SCOREABLE}.$$

143 Prototype Validation Requirement

The Utility Score is an engineering framework for the DisasterOps prototype.
The following quantities require future validation:

- factor weights,
- evaluation horizon,
- system-burden function,
- safety-buffer values,
- lateness-decay constant,
- efficiency reference values,

- feasibility ranking threshold.

Validation should include:

1. synthetic disaster scenarios,
2. historical cases,
3. sensitivity analysis,
4. expert review,
5. SOP review,
6. comparison with human coordinator decisions.

The prototype must clearly state that the numerical coefficients are not yet validated emergency-management standards.

144 Normative Implementation Contracts

144.1 Purpose and Requirement Language

This section converts the logical framework into implementation constraints that a coding agent can execute and test. In this section:

- **MUST** and **MUST NOT** are mandatory for the MVP,
- **SHOULD** and **SHOULD NOT** are recommended unless a documented reason exists,
- **MAY** is optional.

If descriptive prose elsewhere conflicts with a normative contract or invariant in this section, this section takes precedence. No LLM is authorized to relax a MUST rule.

144.2 Decision Context

Every calculation MUST receive and return an immutable decision context:

```
DecisionContext:
  planning_run_id: string
  as_of: UTC timestamp
  state_version: non-negative integer
  config_version: string
  algorithm_version: string
  sop_snapshot_ids: list[string]
```

All candidates in one ranking run MUST use the same `as_of`, `state_version`, `config_version`, and `algorithm_version`. Results from different contexts MUST NOT be mixed in one ranking list.

144.3 Scalar, Unit, and Nullability Conventions

Field type	Canonical representation	Rule
Identifier	Opaque prefixed string	Stable and unique within its object type. Clients MUST NOT parse business meaning from an ID.
Timestamp	ISO 8601 UTC ending in `Z`	Local timestamps without an offset MUST be rejected.
Duration	Integer minutes	All ETA, TTF, delay, and freshness calculations use minutes internally.
Distance	Non-negative integer metres	UI MAY convert to kilometres.
Quantity	Non-negative integer canonical units	Resource configuration defines the canonical unit, for example millilitres, portions, or individual items.
Normalized factor	Decimal in $[0, 1]$	Code MUST clamp only where the equation explicitly requires it. Out-of-range inputs are invalid.
Displayed score	One decimal place plus band	More precision MAY be stored internally but MUST NOT be presented as validated probability.
Unknown value	JSON `null`	Unknown MUST NOT be converted to zero, false, empty string, or an invented default.
Known zero	Numeric zero	Zero is valid evidence and is different from unknown.

Every quantity MUST travel with a resource type or explicit unit. Arithmetic between incompatible units MUST fail validation.

144.4 Core Enumerations

The MVP MUST use closed enumerations at API and persistence boundaries.

```
NeedStatus:
  ACTIVE | STABILIZED | FULFILLED | CANCELLED

RoadStatus:
  OPEN | RESTRICTED | BLOCKED | UNKNOWN

VehicleStatus:
  AVAILABLE | ASSIGNED | IN_TRANSIT | OUT_OF_SERVICE | UNKNOWN
```

```
CalculationStatus:
  CALCULATED | NEEDS_DATA | INVALID_INPUT

CandidateStatus:
  GENERATED | NEEDS_DATA | HARD_INFEASIBLE | SCOREABLE |
  PENDING_APPROVAL | APPROVED | REJECTED | SUPERSEDED | DISPATCHED

ApprovalDecision:
  APPROVE | MODIFY | REJECT

ConfidenceBand:
  LOW | MEDIUM | HIGH
```

Unknown enum strings MUST be rejected at the boundary. Database migrations MUST be used to add new values.

144.5 Urgency Result Contract

```
UrgencyResult:
  schema_version: "1.0"
  context: DecisionContext
  need_id: string
  status: CALCULATED | NEEDS_DATA | INVALID_INPUT
  score_internal: number | null
  score_display: number | null
  band: LOW | MODERATE | HIGH | VERY_HIGH | CRITICAL | null
  factors:
    time_pressure: number | null
    resource_severity: number | null
    vulnerability: number | null
    population: number | null
    lack_of_local_coping: number | null
  ttf_minutes: integer | null
  data_confidence: number
  provisional: boolean
  missing_fields: list[string]
  reason_codes: list[string]
```

`calculateUrgency()` MUST NOT accept or read source ID, vehicle ID, route ID, candidate ID, candidate ETA, or candidate feasibility. For the same need snapshot, evaluation time, and configuration version, the function MUST return the same result regardless of which candidate plans exist.

144.6 Candidate Plan Contract

```
CandidatePlan:
  schema_version: "1.0"
  context: DecisionContext
  candidate_id: string
```

```
  need_id: string
  generation_pattern: DIRECT | ALTERNATIVE_SOURCE | CROSS_SHELTER |
                      DELAYED_VEHICLE | MULTI_TRIP | SPLIT |
                      PARTIAL_STABILIZATION | REASSIGNMENT
  source_ids: list[string]
  destination_shelter_id: string
  deliveries:
    - resource_type: string
      quantity_canonical: integer
      unit: string
      vehicle_id: string
      route_id: string
      depart_at: UTC timestamp
      eta_minutes_from_as_of: integer
  fulfillment_fraction: number
  is_partial: boolean
  assumptions: list[string]
  warnings: list[string]
  status: CandidateStatus
```

Every resource reservation, vehicle assignment, route, and quantity MUST reference an object present in the candidate's exact state version. Partial fulfillment MUST be explicit in both `is_partial` and `fulfillment_fraction`.

144.7 Feasibility Result Contract

```
FeasibilityResult:
  schema_version: "1.0"
  context: DecisionContext
  candidate_id: string
  status: INVALID_INPUT | NEEDS_DATA | HARD_INFEASIBLE | SCOREABLE
  score_internal: number | null
  score_display: number | null
  band: VERY_LOW | LOW | MODERATE | HIGH | VERY_HIGH | null
  factors: object | null
  failed_constraints: list[string]
  missing_fields: list[string]
  uncertainty_codes: list[string]
  explanation_facts: list[string]
```

Only `SCOREABLE` may have a numeric score. All other statuses MUST have `score_internal=null`, `score_display=null`, and `band=null`.

144.8 Utility and Recommendation Contract

```
UtilityResult:
  schema_version: "1.0"
  context: DecisionContext
  candidate_id: string
```

```
  status: CALCULATED | INVALID_INPUT
  score_internal: number | null
  score_display: number | null
  band: string | null
  factors:
    system_impact: number
    effective_coverage: number
    downstream_safety: number
    mission_continuity: number
    operational_efficiency: number
    future_optionality: number
  baseline_state_version: integer
  projected_state_hash: string
  assumptions: list[string]
  uncertainty_codes: list[string]

Recommendation:
  schema_version: "1.0"
  context: DecisionContext
  recommendation_id: string
  need_id: string
  ranked_candidate_ids: list[string]
  recommended_candidate_id: string | null
  no_acceptable_plan: boolean
  reason_codes: list[string]
  expires_at: UTC timestamp
```

Utility MUST be calculated only for `SCOREABLE` candidates. A recommendation MUST expire when its state version is superseded by a material event affecting its need, source, vehicle, route, weather rule, SOP, or reserved inventory.

144.9 Configuration Source of Truth

Every coefficient, threshold, reserve, time scale, display band, hard deadline, and factor weight MUST come from one versioned configuration snapshot. Production code MUST NOT duplicate numeric policy values across modules.

Conceptual structure:

```
DecisionConfig:
  config_version: "prototype-2026-09-13.1"
  urgency:
    weights: {time: 0.40, severity: 0.20, vulnerability: 0.15,
              population: 0.15, local_coping: 0.10}
    resource_time_scales_minutes: object
    resource_time_exponents: object
    severity_table: object
    population_reference: 500
    vulnerability_reference: 0.30
  feasibility:
```

```
    weights: object
    minimum_normal_ranking_score: 6.0
    vehicle_headroom_reference: 0.25
    resource_headroom_reference: 0.50
  utility:
    weights: object
    evaluation_horizon_minutes: 240
    lateness_decay_minutes: 60
  freshness:
    shelter_minutes: integer
    road_minutes: integer
    weather_minutes: integer
```

Application startup MUST validate that every weight vector sums to 1 within tolerance, all scale parameters are positive, all bands are ordered and non-overlapping, and all referenced resource types exist. A planning run MUST retain the exact configuration version used.

144.10 Event Envelope, Idempotency, and Ordering

All external reports and internal state changes MUST use an event envelope:

```
EventEnvelope:
  event_id: globally unique string
  event_type: closed enum
  occurred_at: UTC timestamp
  received_at: UTC timestamp
  actor_type: HUMAN | SYSTEM | AGENT | EXTERNAL_SERVICE
  actor_id: string | null
  municipality_id: string
  object_id: string
  expected_state_version: integer | null
  idempotency_key: string
  payload_schema_version: string
  payload: object
```

Processing the same `idempotency_key` more than once MUST return the first committed result and MUST NOT apply inventory, mission, or approval effects again. Within one municipality, committed state versions MUST increase monotonically.

144.11 Atomic Reservation and Approval Commit

Candidate generation reads inventory but does not consume it. Approval MUST execute one atomic transaction:

1. lock or compare-and-swap every affected inventory and vehicle record,
2. verify the recommendation has not expired,
3. verify `expected_state_version` still matches,

4. rerun all hard constraints against the current transaction snapshot,
5. create reservations,
6. create the mission and approval audit record,
7. commit one new state version.

If any check fails, the transaction MUST roll back with `STALE_RECOMMENDATION` or the applicable constraint code. The system MUST then replan. A coordinator UI MUST NOT dispatch directly from an old client-side candidate object.

144.12 Normative Invariants

ID	Invariant
DATA-01	Unknown operational values MUST remain null and MUST NOT be coerced to zero.
DATA-02	All calculation inputs and outputs MUST carry the same decision context for one planning run.
DATA-03	A stale or offset-less timestamp MUST NOT silently become current UTC time.
URG-01	Urgency MUST depend only on need state, evaluation time, and versioned policy configuration.
URG-02	Changing only source, vehicle, route, or candidate ETA MUST NOT change the need's urgency result.
URG-03	If $TTF \leq 0$, Time Pressure MUST equal 1 and the configured life-safety escalation MUST run.
CG-01	Candidate Generation MUST NOT invent inventory, vehicle capacity, road safety, coordinates, quantity, or approval.
CG-02	Every candidate MUST reference the exact state version from which it was generated.
CG-03	Partial fulfillment MUST never be represented as full fulfillment.
FEAS-01	A required route intersecting a confirmed blocked segment MUST produce `HARD_INFEASIBLE` with null score.
FEAS-02	Missing essential data MUST produce `NEEDS_DATA`, not `HARD_INFEASIBLE` and not a guessed score.
FEAS-03	A transfer MUST leave source inventory at or above reserve after all committed and newly reserved quantities.
FEAS-04	A trip load above vehicle capacity MUST produce `HARD_INFEASIBLE`.
FEAS-05	Pre-authorization MUST NOT create a floor for route, vehicle, timing, weather, resource, or total feasibility.
UTIL-01	Utility MUST be absent for any candidate whose feasibility status is not `SCOREABLE`.
UTIL-02	All candidates in one ranking run MUST use the same baseline state, need cohort, urgency weights, horizon, and configuration.
UTIL-03	Utility MUST NOT compensate for a hard constraint violation.

APP-01	Approval of an expired or stale recommendation MUST fail atomically and trigger replanning.
APP-02	A rejected candidate MUST never create or dispatch a mission.
APP-03	Concurrent approvals MUST NOT reserve the same inventory or vehicle twice.
EVT-01	Reprocessing an event idempotency key MUST produce no additional state mutation.
REP-01	A road closure MUST invalidate every active candidate or mission whose required route intersects the closed segment.
AI-01	The LLM MAY parse, retrieve, orchestrate, and explain; it MUST NOT perform authoritative arithmetic or mutate operational state without validated tools.

144.13 Minimum Acceptance Tests

The following tests are mandatory before a milestone may be considered complete.

```
TEST URG-ETA-SEPARATION
Given one unchanged need snapshot and configuration
And Candidate A has ETA 10 minutes
And Candidate B has ETA 120 minutes
When urgency is calculated
Then both candidates reference exactly the same UrgencyResult
And ETA is absent from calculateUrgency inputs

TEST URG-TIME-ADVANCE
Given TTF is 180 minutes at 14:00Z
When urgency is recalculated at 15:00Z with no delivery
Then TTF is 120 minutes
And intrinsic time pressure increases

TEST UNKNOWN-IS-NOT-ZERO
Given current inventory is null
When a water need is evaluated
Then inventory is not interpreted as zero
And urgency status is NEEDS_DATA if TTF cannot be derived

TEST BLOCKED-ROUTE
Given a candidate route contains segment R17
And R17 status is BLOCKED in the same state version
When feasibility is calculated
Then status is HARD_INFEASIBLE
And score_internal is null
And utility is not calculated

TEST UNKNOWN-ROAD
Given a candidate route contains an UNKNOWN segment
When no SOP prohibits unknown segments
```

```
Then status is not HARD_INFEASIBLE solely for that reason
And route reliability and uncertainty warnings are reduced

TEST DONOR-RESERVE
Given donor on-hand is 500 and committed is 50
And donor reserve is 250
When a candidate transfers 250
Then status is HARD_INFEASIBLE
And no reservation is created

TEST PARTIAL-FULFILLMENT
Given required quantity is 500
And candidate delivers 200
When the candidate is generated
Then is_partial is true
And fulfillment_fraction is 0.4

TEST PREAUTHORIZATION
Given a candidate is PREAUTHORIZED
And its route factor is low but not prohibited
When feasibility is calculated
Then approval readiness is 1
And total feasibility is not floored to 9

TEST STALE-APPROVAL
Given a recommendation was generated from state version 184
And inventory changed at state version 185
When the coordinator approves the old recommendation
Then the transaction returns STALE_RECOMMENDATION
And creates no reservation or mission
And replanning is requested

TEST IDEMPOTENT-REPORT
Given report event E1 has already committed
When E1 is submitted again with the same idempotency key
Then the prior result is returned
And shelter inventory changes exactly zero additional times

TEST REJECTED-PLAN
Given the coordinator rejected Candidate P1
When orchestration continues
Then P1 cannot enter APPROVED or DISPATCHED
And the rejection reason is present in the audit log

TEST COMMON-BASELINE-RANKING
Given Candidates P1, P2, and P3 are ranked together
When their utilities are computed
Then every result has the same planning_run_id
```

```
And baseline_state_version
And evaluation horizon
And configuration version
```

144.14 Implementation Milestones and Completion Gates

The coding agent SHOULD implement vertical milestones in dependency order:

1. shared schemas, enums, configuration validation, and state versioning,
2. report ingestion, validation, duplicate handling, and idempotency,
3. intrinsic Urgency Engine and tests,
4. Candidate Generator with resource and route references,
5. hard-constraint and Feasibility Engine,
6. common-baseline simulation, Utility Engine, and ranking,
7. approval transaction, reservations, missions, and audit log,
8. agent orchestration and structured explanation,
9. volunteer/coordinator UI and road-closure replanning demo.

A milestone is complete only when its unit tests pass, its API contract is validated, migrations are applied, and no later module is required to reinterpret its stored data. The agent MUST NOT begin by generating the entire application from the full document in one unverified step.

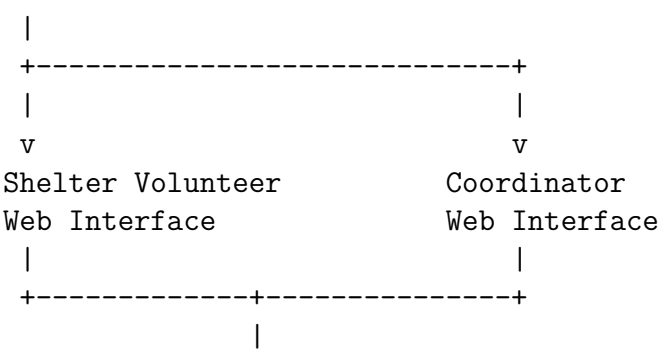
145 Implementation Map and Web Application Architecture

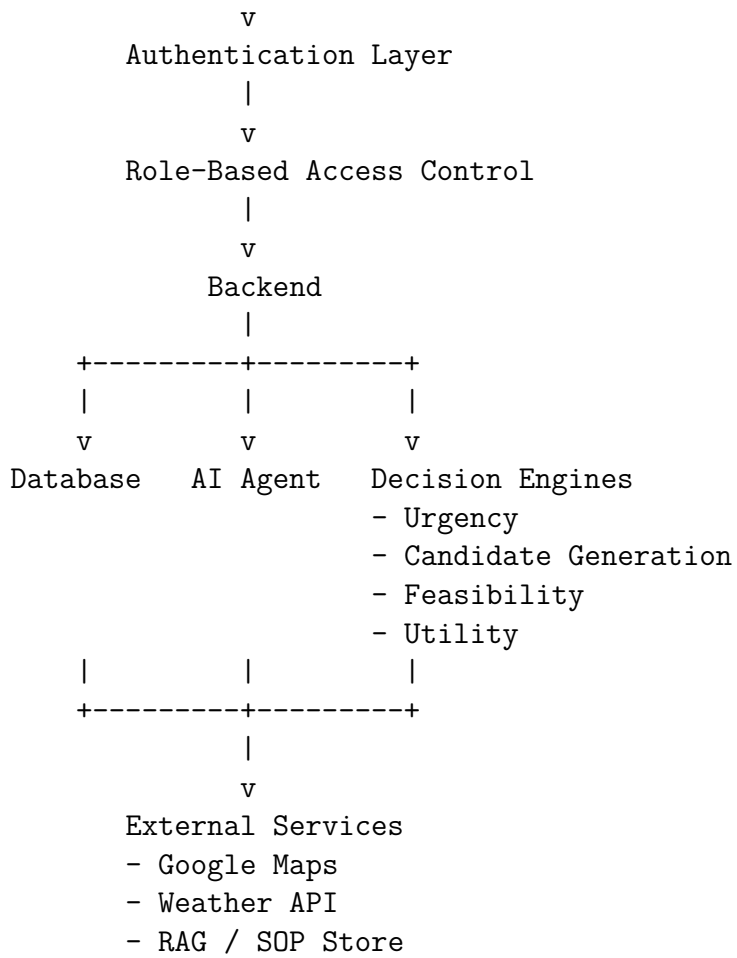
145.1 Purpose

This section converts the DisasterOps logical framework into an implementation-oriented software blueprint.

The prototype is designed as a role-based web application with a shared backend.

The recommended high-level architecture is:





146 Why a Web Application

For the prototype, DisasterOps should be implemented primarily as a responsive web application.

Advantages include:

- no installation required,
- accessible from laptops, tablets, and smartphones,
- rapid deployment during the hackathon,
- centralized updates,
- easier debugging,
- easier integration with backend APIs,
- easier role-based authentication,
- easier demonstration to judges.

The frontend should be responsive so that:

- volunteers can use smartphones,

- coordinators can use tablets or computers.

A future version may optionally become a Progressive Web Application (PWA) to support:

- home-screen installation,
- limited offline operation,
- cached forms,
- background synchronization.

147 User Roles

The initial DisasterOps prototype should support at least two main user roles.

147.1 Role 1: Shelter Volunteer / Shelter Manager

Primary responsibility:

> Report accurate shelter conditions and receive operational instructions.

Volunteer users should not see the full municipal decision dashboard.
They should primarily interact with:

- their assigned shelter,
- hourly status reports,
- emergency updates,
- active requests,
- expected deliveries,
- coordinator instructions.

147.2 Role 2: Municipal Coordinator

Primary responsibility:

> Monitor the full disaster operation and approve or modify DisasterOps recommendations.

Coordinator users should have visibility across:

- all shelters,
- all active needs,
- all vehicles,

- all warehouses,
- road conditions,
- weather,
- AI recommendations,
- active missions,
- approval requests,
- system alerts.

147.3 Optional Future Role: Driver / Logistics Staff

A future implementation may include a Driver role.

Driver functionality may include:

- assigned mission,
- pickup location,
- destination,
- recommended route,
- delivery quantity,
- mission status,
- arrival confirmation,
- road obstruction reporting.

The Driver role is optional for the first hackathon MVP.

148 Authentication

All DisasterOps users should log in.

Prototype authentication flow:

```
Open DisasterOps
       |
       v
Login Page
       |
       v
Username / Email
Password
       |
       v
Authentication
```

```
       |
       v
Load User Role
       |
       +----------------------+
       |                      |
       v                      v
VOLUNTEER                COORDINATOR
       |                      |
       v                      v
Volunteer UI             Coordinator UI
```

148.1 User Object

Conceptual user schema:

```
User:

    user_id

    name

    email

    password_hash

    role

    assigned_shelter_id

    municipality_id

    account_status

    created_at

    last_login
```

Possible roles:

```
VOLUNTEER

SHELTER_MANAGER

COORDINATOR

ADMIN

DRIVER
```

149 Role-Based Access Control

Access should be controlled by role.

For example:

Action	Volunteer	Coordinator
View own shelter	Yes	Yes
View all shelters	No	Yes
Submit hourly report	Yes	Yes
Submit emergency report	Yes	Yes
Create supply request	Yes	Yes
View AI urgency score	Optional	Yes
View all candidate plans	No	Yes
Approve mission	No	Yes
Reject AI recommendation	No	Yes
Reassign vehicle	No	Yes
Modify road status	Limited	Yes
View complete system map	No	Yes

150 Volunteer User Interface

The Volunteer interface should prioritize:

- simplicity,
- speed,
- low cognitive load,
- large buttons,
- minimal typing,
- structured input.

A volunteer should not need to understand:

- utility ranking,
- optimization,
- candidate generation,
- system-wide resource allocation.

150.1 Volunteer Home Screen

Recommended layout:

```
-----------------------------------------------

DISASTEROPS

Shelter A
Last Update: 13:02

STATUS:
Report up to date

Next hourly report:
14:00

-----------------------------------------------

[ UPDATE STATUS ]

[ CRITICAL UPDATE ]

[ REQUEST SUPPLIES ]

-----------------------------------------------

ACTIVE REQUESTS

Water
Status: Approved
Delivery ETA: 34 min

Blankets
Status: Pending Review

-----------------------------------------------

INCOMING DELIVERIES

Van 2
Water: 300 L
ETA: 34 min

-----------------------------------------------

MESSAGES

Coordinator:
```

```
"Please verify secondary road access."

------------------------------------------------
```

151 Volunteer Hourly Status Form

The hourly form should automatically load previous values.
Example:

```
HOURLY STATUS REPORT

Shelter A
14:00

-------------------------------------

POPULATION

Current evacuees:
[ 82 ]

Previous:
80

New arrivals:
[ 4 ]

Departures:
[ 2 ]

-------------------------------------

WATER

Current quantity:
[ 210 L ]

Previous:
260 L

Measurement:
[ Estimated v ]

Last checked:
[ 13:45 ]

-------------------------------------
```

```
FOOD

Meals available:
[ 390 ]

Previous:
430

-------------------------------------

POWER

[ Normal ]
[ Limited ]
[ Failed ]

-------------------------------------

ROAD ACCESS

Primary road:
[ Open ]

Secondary road:
[ Unknown ]

-------------------------------------

Any major change?

[ No ]

[ Yes ]

-------------------------------------

[ SUBMIT REPORT ]

-------------------------------------
```

151.1 Volunteer Critical Update

The Critical Update screen should be extremely simple.

Example:

```
CRITICAL UPDATE

What changed?
```

```
[ RESOURCE SHORTAGE ]

[ ROAD BLOCKED ]

[ POWER FAILURE ]

[ SHELTER ACCESS PROBLEM ]

[ RAPID POPULATION INCREASE ]

[ OTHER ]

--------------------------------

Describe:

[ _____ ]

Optional Photo:

[ UPLOAD ]

Optional Map Location:

[ ADD LOCATION ]

--------------------------------

[ SEND CRITICAL UPDATE ]
```

Submitting a Critical Update creates a new event immediately. It should not wait for the next hourly reporting cycle.

152 Volunteer Supply Request Screen

Example:

```
REQUEST SUPPLIES

Resource:
[ Drinking Water v ]

Quantity requested:
[ 300 L ]

Current amount:
[ 150 L ]
```

```
Reason:
[ Stock decreasing faster than expected ]

Human perceived urgency:

[ Low ]
[ Medium ]
[ High ]
[ Critical ]

--------------------------------

[ SUBMIT REQUEST ]
```

Human urgency is not directly used as the final DisasterOps urgency score. It is treated as an additional signal.

153 Volunteer Delivery Screen

Volunteers should be able to see only deliveries relevant to their shelter.

Example:

```
INCOMING DELIVERY

Resource:
Drinking Water

Quantity:
300 L

Vehicle:
Van 2

Status:
IN TRANSIT

ETA:
34 min

--------------------------------

When delivery arrives:

[ CONFIRM DELIVERY ]

Actual quantity received:
[ 300 L ]
```

```
Damaged / Missing:
[ 0 L ]

[ COMPLETE ]
```

154 Coordinator User Interface

The Coordinator interface should be significantly more information-dense.
Its purpose is operational decision-making rather than data entry.
The coordinator dashboard should contain:

- shelter overview,
- map,
- active requests,
- urgency ranking,
- AI recommendations,
- active missions,
- vehicle state,
- warehouses,
- road status,
- weather alerts,
- system warnings.

154.1 Coordinator Dashboard

Conceptual layout:

```
============================================================

DISASTEROPS COMMAND DASHBOARD

Active Shelters: 5
Critical Needs: 2
Active Missions: 4
Vehicles Available: 2 / 5

============================================================

MAP

[ Shelters ]
```

[Warehouses]
[Vehicles]
[Active Routes]
[Road Closures]
[Weather Alerts]

==========

PRIORITY NEEDS

1. Shelter A
   Water
   Urgency: 9.1

2. Shelter C
   Power
   Urgency: 8.7

3. Shelter D
   Food
   Urgency: 6.8

==========

AI RECOMMENDATION

Target:
Shelter A

Recommended Plan:
P1

Utility:
9.0 / 10

Feasibility:
9.3 / 10

Source:
Central Warehouse

Vehicle:
Van 1

ETA:
35 min

[VIEW REASONING]

```
[ APPROVE ]

[ MODIFY ]

[ REJECT ]

============================================================

ACTIVE MISSIONS

Van 2 -> Shelter B
Status: IN TRANSIT

Van 4 -> Shelter D
Status: DELAYED

============================================================
```

155 Coordinator Map

The map is a central component of the Coordinator UI.

Map layers may include:

- shelter locations,
- warehouse locations,
- vehicle positions,
- active routes,
- alternative routes,
- blocked roads,
- restricted roads,
- weather warnings,
- active incidents.

155.1 Map Interaction

Selecting a shelter should open:

```
Shelter A

Population:
82
```

```
Water:
210 L

Water TTF:
2.3 hours

Urgency:
9.1

Active Requests:
2

Data Confidence:
91%

Last Update:
13:45

[ VIEW DETAILS ]
```

156 Coordinator AI Recommendation Screen

The recommendation screen should show more than a score.

Example:

```
RECOMMENDED PLAN P1

Target:
Shelter A

Need:
300 L Drinking Water

Urgency:
9.1 / 10

--------------------------------

PLAN

Source:
Central Warehouse

Vehicle:
Van 1

Route:
Route A
```

ETA:
35 min

FEASIBILITY:
9.3 / 10

UTILITY:
9.0 / 10

WHY THIS PLAN?

+ Full need coverage
+ Arrives before predicted shortage
+ No donor shelter endangered
+ No critical mission interrupted
+ Route currently open
+ Weather risk low
+ Preserves 2 vehicles for future incidents

ALTERNATIVES

P2
Utility: 7.9

Reason lower:
Requires vehicle reassignment

P3
Utility: 7.1

Reason lower:
Reduces Shelter B reserve

[APPROVE]

[MODIFY]

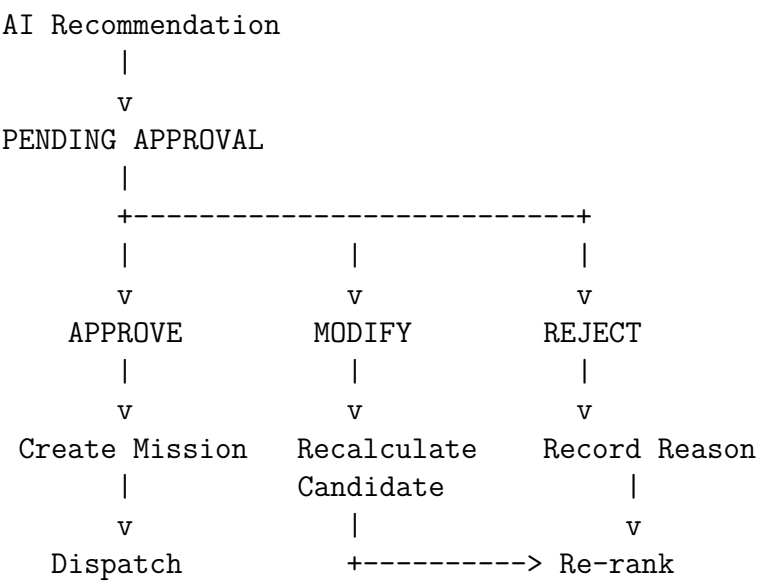
[SELECT ALTERNATIVE]

```
[ REJECT ]

------------------------------
```

157 Coordinator Approval Workflow

Approval state machine:



158 Coordinator Modify Plan

The coordinator may manually modify:

- source,
- quantity,
- vehicle,
- route,
- departure time.

However, manual modifications must still pass the deterministic safety system. For example:

```
Coordinator changes:

Vehicle:
Van 1 -> Van 4

       |
       v
```

```
DisasterOps recalculates:

Capacity
ETA
Route
Feasibility
Utility
```

A human should not accidentally bypass hard constraints through the UI.

159 Coordinator Shelter Detail Page

Coordinator Shelter Detail should contain:

- current population,
- vulnerable population,
- inventory,
- resource runway,
- infrastructure,
- road access,
- requests,
- incoming deliveries,
- historical reports,
- data confidence,
- active alerts.

160 Frontend Route Structure

Possible web routes:

```
/login

/volunteer/dashboard

/volunteer/report

/volunteer/critical-update

/volunteer/request

/volunteer/deliveries
```

```
/coordinator/dashboard

/coordinator/map

/coordinator/shelters

/coordinator/shelters/:id

/coordinator/needs

/coordinator/recommendations

/coordinator/missions

/coordinator/vehicles

/coordinator/resources

/coordinator/roads

/coordinator/alerts
```

161 Frontend Component Map

Possible frontend components:

```
Authentication

    LoginForm
    ProtectedRoute
    RoleGuard


Volunteer

    VolunteerDashboard
    HourlyReportForm
    CriticalUpdateForm
    SupplyRequestForm
    DeliveryCard
    MessagePanel


Coordinator

    CoordinatorDashboard
```

```
    OperationsMap
    ShelterList
    ShelterDetail
    PriorityQueue
    RecommendationCard
    CandidateComparison
    MissionList
    VehiclePanel
    WarehousePanel
    RoadAlertPanel
    WeatherAlertPanel
```

162 Backend Module Architecture

Recommended backend modules:

```
auth/

users/

shelters/

reports/

resources/

requests/

vehicles/

roads/

weather/

routing/

missions/

approvals/

rag/

agent/

decision/
    urgency/
    candidate_generation/
    feasibility/
```

```
    utility/

notifications/

audit/
```

163 Core Backend Function Map

Function	Purpose
validateReport()	Validate incoming shelter report
detectDuplicate()	Detect duplicate or overlapping requests
detectConflict()	Detect conflicting data
updateSharedState()	Update operational state
calculateUrgency()	Calculate need urgency
generateCandidates()	Generate response plans
calculateFeasibility()	Evaluate candidate executability
simulatePlan()	Project system state after candidate
calculateUtility()	Evaluate system-wide candidate value
rankPlans()	Rank candidate plans
retrieveSOP()	Retrieve relevant rules
computeRoutes()	Obtain map routes and ETA
getWeather()	Obtain route weather information
createMission()	Create approved operational mission
monitorMission()	Track active mission
replanAffectedMissions()	Respond to changed state

164 API Endpoint Map

Possible backend REST API design:

```
POST /api/auth/login

GET  /api/me


GET  /api/shelters

GET  /api/shelters/:id

POST /api/shelters/:id/reports

POST /api/shelters/:id/critical-updates

POST /api/shelters/:id/requests


GET  /api/needs
```

```
GET  /api/needs/:id

POST /api/needs/:id/recalculate


GET  /api/recommendations/:need_id

POST /api/recommendations/:id/approve

POST /api/recommendations/:id/reject

POST /api/recommendations/:id/modify


GET  /api/missions

GET  /api/missions/:id

POST /api/missions/:id/status


GET  /api/vehicles

GET  /api/resources

GET  /api/roads

POST /api/roads/events


GET  /api/weather

GET  /api/routes


GET  /api/audit
```

All mutating endpoints MUST require an `Idempotency-Key`. Approval, rejection, modification, mission-status, and road-event commands MUST also provide `expected_state_version`. A stale version returns HTTP `409` with machine-readable code `STALE_STATE`; validation failures return `422`; missing authorization returns `403`. Error responses MUST NOT contain an invented fallback result.

165 Example Shelter Report JSON

Conceptual request:

```
{
```

```
  "schema_version": "1.0",
  "report_id": "REPORT_20261107_1400_A",
  "idempotency_key": "device7-20261107-1400-water",
  "shelter_id": "SHELTER_A",
  "occurred_at": "2026-11-07T14:00:00Z",

  "population": {
    "total": 82,
    "new_arrivals": 4,
    "departures": 2
  },

  "vulnerable_groups": {
    "elderly": 12,
    "mobility_assistance": 3,
    "pregnant": 1,
    "young_children": 5
  },

  "resources": [
    {"type": "WATER", "quantity_canonical": 210000,
     "unit": "MILLILITRE", "measurement": "ESTIMATED"},
    {"type": "FOOD", "quantity_canonical": 390,
     "unit": "PORTION", "measurement": "COUNTED"},
    {"type": "BLANKET", "quantity_canonical": 95,
     "unit": "ITEM", "measurement": "COUNTED"},
    {"type": "BATTERY", "quantity_canonical": 32,
     "unit": "ITEM", "measurement": "COUNTED"}
  ],

  "infrastructure": {
    "electricity": "NORMAL",
    "water_supply": "LIMITED",
    "sanitation": "NORMAL",
    "mobile_network": "WEAK"
  },

  "road_access": {
    "primary": "OPEN",
    "secondary": "UNKNOWN"
  },

  "major_change": false
}
```

166 Example Need Object

```
{
```

```
  "schema_version": "1.0",
  "need_id": "NEED_104",
  "shelter_id": "SHELTER_A",
  "resource_type": "WATER",
  "unit": "MILLILITRE",
  "current_quantity_canonical": 210000,
  "target_quantity_canonical": 500000,
  "consumption_per_hour_canonical": 80000,
  "time_to_failure_minutes": 158,
  "urgency_result_id": "URGENCY_104_V184",
  "state_version": 184,
  "data_confidence": 0.91,
  "status": "ACTIVE"
}
```


167 Example Candidate Plan Object


```
{
  "schema_version": "1.0",
  "planning_run_id": "RUN_104_V184",
  "state_version": 184,
  "config_version": "prototype-2026-09-13.1",
  "candidate_id": "PLAN_18",
  "need_id": "NEED_104",
  "generation_pattern": "DIRECT",
  "source_ids": ["WH_01"],
  "destination_shelter_id": "SHELTER_A",
  "deliveries": [{
    "resource_type": "WATER",
    "quantity_canonical": 300000,
    "unit": "MILLILITRE",
    "vehicle_id": "VAN_01",
    "route_id": "R_882",
    "depart_at": "2026-11-07T14:05:00Z",
    "eta_minutes_from_as_of": 35,
    "distance_metres": 14200
  }],
  "fulfillment_fraction": 1.0,
  "is_partial": false,
  "weather_risk": 0.12,
  "feasibility_result_id": "FEAS_18_V184",
  "utility_result_id": "UTILITY_18_V184",
  "approval_required": true,
  "status": "SCOREABLE"
}
```

168 Example Recommendation Object


```
{
  "schema_version": "1.0",
  "planning_run_id": "RUN_104_V184",
  "state_version": 184,
  "config_version": "prototype-2026-09-13.1",
  "recommendation_id": "REC_104_V184",
  "need_id": "NEED_104",
  "recommended_candidate_id": "PLAN_18",
  "no_acceptable_plan": false,
  "ranking": [
    {
      "candidate_id": "PLAN_18",
      "utility_display": 9.0,
      "feasibility_display": 9.3
    },
    {
      "candidate_id": "PLAN_21",
      "utility_display": 7.9,
      "feasibility_display": 8.6
    },
    {
      "candidate_id": "PLAN_24",
      "utility_display": 7.1,
      "feasibility_display": 8.8
    }
  ],
  "explanation": {
    "full_coverage": true,
    "arrives_before_failure": true,
    "downstream_risk": "LOW",
    "mission_disruption": "NONE",
    "route_status": "OPEN",
    "weather_risk": "LOW",
    "uncertainty_codes": []
  },
  "expires_at": "2026-11-07T14:10:00Z"
}
```


169 Database Tables

A relational database may contain the following tables:

`users`

`municipalities`

```
shelters

shelter_reports

shelter_population

shelter_resources

resource_requests

warehouses

warehouse_inventory

vehicles

drivers

road_segments

road_events

weather_snapshots

needs

candidate_plans

candidate_routes

recommendations

missions

mission_events

approvals

sop_documents

audit_logs
```

170 Core Database Relationships

Conceptually:

```
Municipality
    |
```

```
    +---- Shelters
    |
    +---- Warehouses
    |
    +---- Vehicles
    |
    +---- Users


Shelter
    |
    +---- Reports
    |
    +---- Resources
    |
    +---- Requests
    |
    +---- Needs


Need
    |
    +---- Candidate Plans
             |
             +---- Routes
             |
             +---- Feasibility
             |
             +---- Utility


Recommendation
    |
    +---- Selected Candidate
             |
             v
           Mission
             |
             +---- Mission Events
```

171 Shared State Service

The database stores persistent information.

However, the decision system should construct a current operational state:

$$X_t.$$

Conceptually:

```
getSharedState():

    shelters

    shelter inventory

    active needs

    warehouses

    available supplies

    vehicles

    drivers

    active missions

    road state

    weather state

    approvals

    current time

RETURN X_t
```

Every decision calculation should reference a state version.
Example:

```
state_version = 184
```

172 Decision Engine API

The central planning endpoint may conceptually work as:

```
POST /api/needs/:id/plan
```

Internal pipeline:

```
need =
    getNeed()

state =
    getSharedState()

context =
    createDecisionContext(
```

```
        state.version,
        current_config.version,
        current_algorithm.version,
        now_utc
    )

urgency =
    calculateUrgency(
        need,
        state,
        context
    )

IF urgency.status != CALCULATED:
    RETURN structured missing-data or invalid-input result

candidates =
    generateCandidates(
        need,
        state
    )

FOR candidate:

    candidate.feasibility =
        calculateFeasibility(
            candidate,
            state,
            context
        )

PARTITION candidates by feasibility status
Only SCOREABLE continues to utility

B_before =
    calculateSystemBurden(
        common_baseline_state,
        frozen_need_cohort,
        frozen_urgency_weights
    )

FOR candidate:

    projected_state =
        simulatePlan(
            candidate,
            state,
            context
```

```
        )

    candidate.utility =
        calculateUtility(
            candidate,
            common_baseline_state,
            projected_state,
            B_before,
            context
        )


ranked =
    rankPlans(candidates)


RETURN:
    ranked plans
    explanations
    decision context
    excluded candidate statuses
```

173 Agent Orchestration Layer

The AI Agent should coordinate tools rather than directly calculate all variables.

Conceptual tools:

```
Tool:
getShelterState()

Tool:
getInventory()

Tool:
calculateUrgency()

Tool:
retrieveSOP()

Tool:
computeRouteMatrix()

Tool:
computeRoutes()

Tool:
getWeather()
```

```
Tool:
generateCandidates()

Tool:
calculateFeasibility()

Tool:
simulatePlan()

Tool:
calculateUtility()

Tool:
createApprovalRequest()

Tool:
createMission()

Tool:
sendNotification()
```

173.1 Example Agent Workflow

```
Agent receives:

"Shelter A urgently needs water."

        |
        v

Call:
getShelterState(A)

        |
        v

Missing quantity?

YES

        |
        v

Send request:
"Please confirm current water quantity."

        |
        v
```

```
Updated report received

        |
        v

Call:
calculateUrgency()

        |
        v

Urgency = 8.9

        |
        v

Call:
generateCandidates()

        |
        v

Candidate set generated

        |
        v

Call:
calculateFeasibility()
for each candidate

        |
        v

Call:
simulatePlan()

        |
        v

Call:
calculateUtility()

        |
        v

Rank plans
```

```
        |
        v

Send recommendation
to coordinator
```

174 External Service Integration Map

External Service	Use
Google Maps Routes API	Routing, distance, ETA, traffic-aware routes, alternative routes
Weather API	Current weather, forecast, weather-related operational risk
RAG Knowledge Base	Retrieve disaster SOPs and operational rules
LLM API	Natural-language interpretation, tool selection, reasoning explanation

175 Event-Driven Replanning Architecture

Changes should create events.

Examples:

```
ROAD_BLOCKED

VEHICLE_FAILURE

NEW_SHELTER_REPORT

NEW_CRITICAL_REQUEST

INVENTORY_CHANGED

WEATHER_ALERT

MISSION_DELAYED

MISSION_COMPLETED

HUMAN_REJECTED_PLAN
```

Event flow:

```
New Event
    |
    v
Update Database
```

```
    |
    v
Update Shared State
    |
    v
Identify affected:
Shelters
Needs
Missions
Routes
    |
    v
Recalculate only affected logic
    |
    v
Generate replacement recommendation
if necessary
```

Events MUST be processed through the normative event envelope. Dependency matching SHOULD use explicit object IDs and route-segment IDs rather than LLM semantic judgment. Replanning jobs MUST be idempotent and MUST verify that the triggering state version is still current before publishing a recommendation.

176 Notification System

Volunteer notifications may include:

- hourly report reminder,
- additional information request,
- request approved,
- delivery dispatched,
- delivery delayed,
- coordinator message.

Coordinator notifications may include:

- critical new request,
- urgency above threshold,
- road closure,
- failed mission,
- recommendation requiring approval,
- stale shelter data,
- no feasible plan found.

177 Low Connectivity and Offline Behaviour

Disasters may reduce internet connectivity.

The web application should therefore be designed so that a future PWA version may support:

- locally cached latest shelter state,
- locally stored incomplete form,
- queued report submission,
- synchronization when connection returns.

A report stored offline should include:

```
created_at

device_timestamp

sync_status
```

Example:

```
sync_status:
PENDING
```

When connectivity returns:

```
PENDING
    |
    v
UPLOAD
    |
    v
VALIDATE
    |
    v
MERGE / CONFLICT CHECK
```

178 Data Conflict After Offline Synchronization

Example:

```
Volunteer offline report:

Water = 200 L
Created 14:05
```

```
Coordinator update:

Water = 250 L
Created 14:20
```

When the offline report synchronizes at:

$$14:30,$$

the system should not blindly overwrite newer data.
Instead:

```
Detect timestamp conflict
        |
        v
Compare sources
        |
        v
Flag discrepancy
        |
        v
Request confirmation
```

179 Audit Logging

Every important decision should be recorded.
Audit events should include:

- report submitted,
- state changed,
- urgency calculated,
- candidate generated,
- feasibility calculated,
- plan ranked,
- recommendation created,
- coordinator approved,
- coordinator modified,
- coordinator rejected,
- mission created,
- mission replanned.

Conceptual audit object:

```
AuditLog:

    event_id
    idempotency_key

    event_type

    actor_type

    actor_id

    occurred_at
    recorded_at

    state_version_before

    state_version_after

    object_id

    planning_run_id
    config_version
    algorithm_version
    sop_snapshot_ids

    input_hash
    output_hash

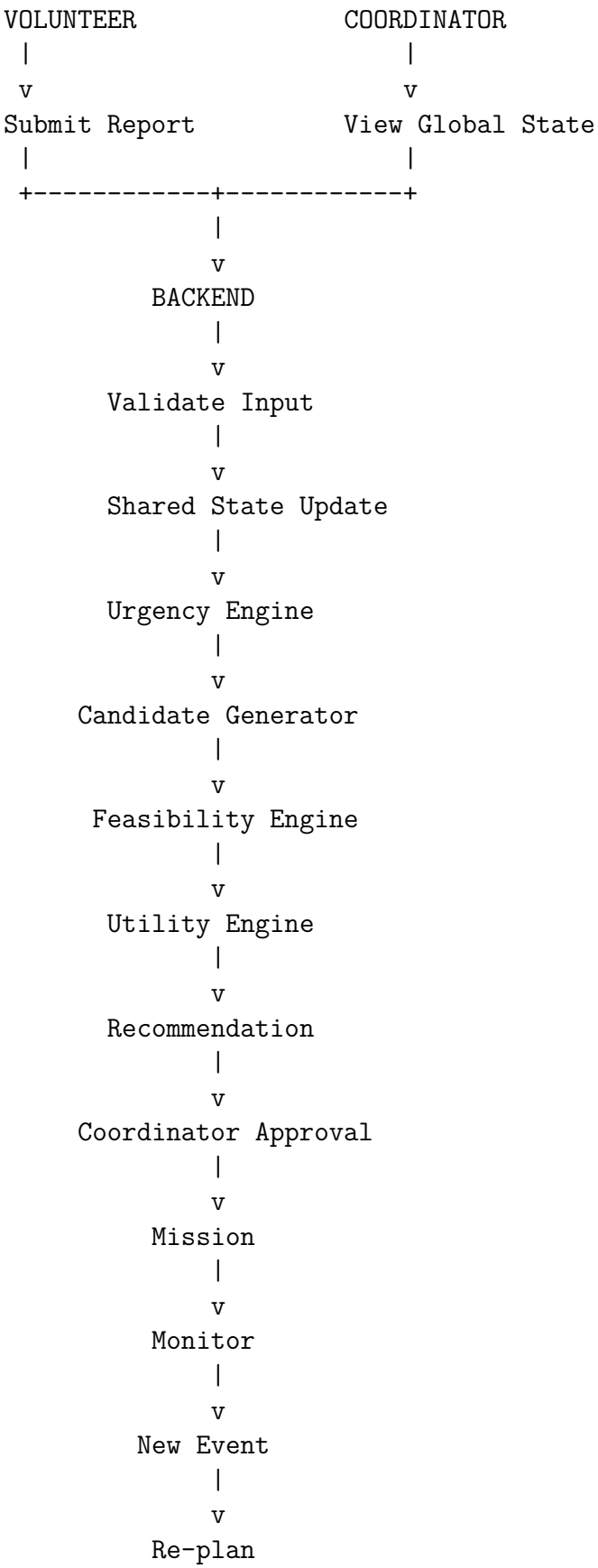
    reason_codes
    explanation_facts
    llm_trace_reference
```

Audit records MUST be append-only. Natural-language explanation is supporting information; reason codes, version identifiers, hashes, and structured state transitions are authoritative for reproduction.

180 System Status Flow

The overall software flow is:

```
LOGIN
 |
 v
ROLE CHECK
 |
 +-------------------------+
 |                         |
 v                         v
```


VOLUNTEER
COORDINATOR
Submit Report
View Global State
BACKEND
Validate Input
Shared State Update
Urgency Engine
Candidate Generator
Feasibility Engine
Utility Engine
Recommendation
Coordinator Approval
Mission
Monitor
New Event
Re-plan

181 Suggested Prototype Technology Stack

One possible hackathon implementation is:

```
Frontend:

React
or
Next.js


Backend:

Python FastAPI
or
Node.js


Database:

PostgreSQL

For very early prototype:
SQLite is acceptable


AI / Agent:

Existing LLM API


RAG:

Embedding model
+
Vector database
+
Keyword retrieval


Routing:

Google Maps Platform Routes API


Weather:

Weather API
```

```
Deployment:

Cloud web deployment
```

The exact implementation stack may be changed by the development team.
The logical architecture should remain independent from a specific framework.

182 Minimum Hackathon Implementation

A minimal functional prototype does not require every database field or UI screen described in this specification.

The MVP should prioritize:

1. login,
2. Volunteer Dashboard,
3. hourly report submission,
4. Critical Update,
5. Coordinator Dashboard,
6. several simulated shelters,
7. resource inventory,
8. vehicle state,
9. map,
10. Urgency Engine,
11. Candidate Generator,
12. Feasibility Engine,
13. Utility Ranking,
14. Top-3 recommendation display,
15. human approval,
16. simulated road closure,
17. automatic replanning.

183 Recommended Demo User Flow

```
VOLUNTEER LOGIN

Shelter A volunteer submits:

Water:
210 L

Population:
82

Road:
Open

       |
       v

DisasterOps calculates:

Water need urgency:
8.7

       |
       v

CANDIDATE GENERATION

P1:
Warehouse + Van 1

P2:
Shelter B transfer

P3:
Warehouse 2 + Van 3

       |
       v

FEASIBILITY

P1:
9.3

P2:
8.8
```

P3:
8.4

|
v

UTILITY

P1:
9.0

P2:
7.1

P3:
8.0

|
v

COORDINATOR DASHBOARD

Recommended:
P1

|
v

Coordinator:
APPROVE

|
v

Mission created

|
v

EVENT:

Volunteer reports:
Road blocked

|
v

DisasterOps:

```
Invalidate old route

Generate replacement candidates

Query Google Maps

Check weather

Recalculate feasibility

Recalculate utility

       |
       v

NEW RECOMMENDATION

       |
       v

Coordinator approves new route
```

184 Final Implementation Principle

The two interfaces serve different operational purposes.

Volunteer UI

$$\boxed{Simple + Fast + LocalShelterInformation}$$

Coordinator UI

$$\boxed{GlobalState + DecisionSupport + Approval + Monitoring}$$

Both interfaces operate on the same shared backend state.
Therefore:

$$\boxed{DifferentUI \quad + \quad SameOperationalTruth}$$

This role separation is a core design principle of DisasterOps.